

PROJECT REPORT

# CreaselQ

---

|               |                                           |
|---------------|-------------------------------------------|
| Project Title | CreaselQ: IPL Match Intelligence Platform |
|---------------|-------------------------------------------|

|              |                 |
|--------------|-----------------|
| Submitted by | Jyotshna Payasi |
|--------------|-----------------|

|                     |            |
|---------------------|------------|
| Registration number | 26BCE10651 |
|---------------------|------------|

|         |                          |
|---------|--------------------------|
| Faculty | Dr. Pradeep Kumar Mishra |
|---------|--------------------------|

|                 |                   |
|-----------------|-------------------|
| Submission date | 30 September 2026 |
|-----------------|-------------------|

# Contents

---

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

## 2. Introduction

---

### 2.1 Context

The Indian Premier League (IPL) gives cricket fans plenty to discuss, from toss decisions to team form. The dataset used in this project contains 1,243 matches across 19 seasons, starting in 2008. We used these records to explore questions that come up every season:

- Does winning the toss decide the match?
- Is chasing easier?
- Did the 2023 *Impact Player* rule change the game?
- Can anyone predict who will win?

The match records can help answer these questions, but they need careful cleaning. We also need to show how certain, or uncertain, each conclusion is.

### 2.2 What CreaselQ is

We built CreaselQ to turn the supplied IPL match file into a useful analysis and prediction tool. It has five main modules:

1. Data engineering (M1). Checks and cleans the raw records, standardises names, and stores the data in a relational database. A quality report records each correction.
2. Analytics and visualisation (M2). Lets users explore the data through an interactive dashboard, with statistical tests and confidence intervals to support the findings.
3. Prediction engine (M3). Estimates each team's chance of winning before a match. It uses only information available at prediction time and explains the result.
4. Scenarios (M4). Explores toss, venue and opponent changes, and simulates a season.
5. Operations and reporting (M5). Keeps logs, records predictions, checks for changes in the data, and produces the model card, figures and verification reports.

### 2.3 Objectives

1. Check and clean the dataset, while keeping a clear record of every change.
2. Store the data in a structured database that can be rebuilt with the same results.
3. Study toss, venue, team and era effects, with uncertainty reported alongside the findings.
4. Estimate match outcomes using past information, time-ordered validation and explained probabilities.
5. Provide an easy-to-use dashboard and command-line interface, supported by logging and tests.
6. Explain the design and results using the artefacts required for the assignment.

### 2.4 Relevance to the course

The project brings together several topics from the machine-learning course:

- **Data preprocessing:** schema validation, anomaly correction, imputation.
- **Exploratory and inferential statistics.**
- **Feature engineering:** including sequential Elo ratings.

- **Supervised classification and regression:** logistic regression, random forests, gradient boosting and ridge regression.
- **Unsupervised learning:** k-means venue clustering.
- **Model selection and evaluation for time-ordered data:** walk-forward CV, the one-standard-error rule, bootstrap confidence intervals and paired tests.
- **Probability calibration and interpretability.**
- **ML ethics:** data leakage, honest reporting, a betting disclaimer.

The project also connects to the course's AI fundamentals. Hyperparameter tuning is an example of heuristic search, the YAML identity rules form a small knowledge base, and the prediction service acts as a simple agent. The file docs/course\_mapping.md links these topics to the relevant code and tests.

## 2.5 Structure of this report

Sections 3-5 explain the problem and requirements. Sections 6-8 describe the architecture, diagrams and design choices. Section 9 covers the data, cleaning, features and evaluation method. Results and screenshots appear in Section 10, followed by testing in Section 11. Sections 12-14 discuss challenges, lessons and future improvements. References are listed in Section 15.

Main finding. Predicting IPL winners from pre-match information proved difficult. The best walk-forward log-loss was 0.6857, compared with 0.6931 for a 50:50 forecast. On the 2025-26 holdout, no selected model performed better than that baseline. We kept this result in the report because it shows the limits of the available data. Very high reported accuracy needs careful checks for data leakage (Section 8).

## 3. Problem Statement

### 3.1 The problem

IPL records contain useful information, but several inconsistencies can affect an analysis:

- Franchise names change. Delhi Daredevils became Delhi Capitals, while Deccan Chargers and Sunrisers Hyderabad need to be treated as separate franchises.
- Venue names vary. The same stadium may appear with different spellings or with its old and new names.
- Some season labels do not match a single calendar year.
- A column's meaning may change over time without an obvious warning.

Students, analysts and fans need data they can trust before comparing teams, venues or toss decisions. Prediction adds another challenge: using results or later matches during training can make a model look more accurate than it really is. CreaseIQ addresses both problems by cleaning the records and testing predictions in time order.

The project therefore focuses on three outcomes:

1. A consistent dataset with a traceable record of corrections.
2. Clear analysis that explains the uncertainty in each finding.
3. Pre-match win probabilities evaluated without using information from the future.

### 3.2 Data traps discovered (and handled)

Table 1: Data problems found during profiling and research, and how they are handled

| Trap                                                    | Evidence                                                                                                  | Handling                                                                                   |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <code>team1</code> means "batting first" only from 2018 | share of matches with raw <code>team1</code> batting first: 1.00 every season from 2018; 0.45-0.62 before | Batting order is derived from the toss; orientation uses a hash-seeded coin flip (ADR-005) |
| Tie rows include super-over runs and wickets            | 16 ties; e.g. a side "losing 12 wickets" (10 + 2 in the super over)                                       | Regulation scores restored from verified external data (ADR-004)                           |
| Voided and replayed match stored as "no result"         | 1 match (08-05-2025, security blackout)                                                                   | <code>voided</code> flag; excluded from scoring statistics                                 |
| D/L-affected results                                    | 14 flagged by margin logic, 16 from a verified list                                                       | <code>dls_flag</code> = heuristic OR list (19 rows)                                        |
| 19 team strings for 15 franchises                       | renames and spelling variants                                                                             | Lineage map with validity years (ADR-003)                                                  |
| 60 venue strings for 37 venues                          | suffixes, renames, rebuilt stadiums                                                                       | Canonical venue map, fail-loud                                                             |
| City "Unknown" in 51 rows                               | Dubai and Sharjah venues                                                                                  | Imputed from the canonical venue                                                           |

### 3.3 Scope

In scope:

- The supplied match file (2008-2026) and validated uploads of new matches.
- A relational database.
- Analytics: toss, chasing, teams, venues, scoring eras, player-of-the-match.

- Two-tier win probability: pre-toss and post-toss.
- First-innings score regression.
- What-if scenarios and a season simulation.
- A dashboard, CLI, tests, CI and documentation.

**Out of scope:**

- Ball-by-ball or live in-play prediction.
- Betting odds or betting advice.
- Player batting and bowling statistics, which are not in the data.
- Multi-user accounts. The current application runs locally for one user and stores public match records. Account management is outside this version's scope.

### 3.4 Target users

*Table 2: Target users and their needs*

| User                         | Need                                                    | Served by                                                                            |
|------------------------------|---------------------------------------------------------|--------------------------------------------------------------------------------------|
| Students / analysts          | Clean data; honest statistics with confidence intervals | Data Explorer, Team Analytics, Venue & Toss, CSV export                              |
| Cricket fans                 | Evidence-based previews; "what if we win the toss?"     | Predict Match, What-If                                                               |
| Maintainers / data engineers | Add seasons and sources safely; retrain reproducibly    | CLI ( <code>ingest</code> , <code>all</code> , <code>train</code> ), tests, CI, ADRs |

## 4. Functional Requirements

CreaseIQ has five main modules (M1-M5) and 22 functional requirements, exceeding the assignment's minimum of three modules. Requirement IDs appear in code and tests, with the full mapping in

`docs/rubric_traceability.md`.

*Table 3: Functional requirements with priority, implementation and verifying tests*

| ID    | Module | Requirement                                                                                            | Pri. | Implementation                                                                  | Test                                                           |
|-------|--------|--------------------------------------------------------------------------------------------------------|------|---------------------------------------------------------------------------------|----------------------------------------------------------------|
| FR-01 | M1     | Ingest CSV; schema and type enforcement; quarantine bad rows with reasons                              | P0   | <code>data/source.py</code> , <code>ingest.py</code> , <code>schema.py</code>   | <code>test_ingest.py</code>                                    |
| FR-02 | M1     | Clean and canonicalise teams, venues, cities, seasons; derive batting order, stage, champion, D/L, era | P0   | <code>data/cleaning.py</code> , <code>canonical.py</code>                       | <code>test_cleaning.py</code> , <code>test_canonical.py</code> |
| FR-03 | M1     | Data-quality report (MD + JSON) and data dictionary                                                    | P0   | <code>data/quality_report.py</code>                                             | <code>test_quality_report.py</code>                            |
| FR-04 | M1     | Normalised SQLite store and Parquet cache; idempotent rebuild                                          | P0   | <code>db/models.py</code> , <code>loader.py</code> , <code>repository.py</code> | <code>test_db.py</code>                                        |
| FR-05 | M1     | Validated CSV append (dry run, dedup, size and type limits)                                            | P1   | <code>services/ingest_service.py</code>                                         | <code>test_services.py</code>                                  |
| FR-06 | M2     | Team records, titles, season form, head-to-head, rating timeline                                       | P0   | <code>analytics/team_stats.py</code>                                            | <code>test_analytics.py</code>                                 |
| FR-07 | M2     | Venue analytics with shrinkage; toss habits; clustering                                                | P0   | <code>analytics/venue_stats.py</code> , <code>venue_clusters.py</code>          | <code>test_analytics.py</code>                                 |
| FR-08 | M2     | Toss analysis with hypothesis tests and CIs; era comparison                                            | P0   | <code>analytics/toss_analysis.py</code> , <code>hypothesis_tests.py</code>      | <code>test_hypothesis_tests.py</code>                          |
| FR-09 | M2     | Scoring and margin trends; closest finishes                                                            | P1   | <code>analytics/season_trends.py</code>                                         | <code>test_analytics.py</code>                                 |
| FR-10 | M2     | Player-of-the-match leaderboard; appearances; squad continuity                                         | P1   | <code>analytics/player_stats.py</code>                                          | <code>test_analytics.py</code>                                 |
| FR-11 | M2     | Interactive filters and sanitised CSV export                                                           | P0   | <code>services/analytics_service.py</code> , <b>app</b><br>page 1               | <code>test_services.py</code> , <code>test_app.py</code>       |

| ID    | Module | Requirement                                                                  | Pri. | Implementation                                                                                                  | Test                                                             |
|-------|--------|------------------------------------------------------------------------------|------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| FR-12 | M3     | Leakage-safe feature builder with an <code>as_of</code> contract             | P0   | <code>features/builder.py</code> (+ 4 state modules)                                                            | <code>test_leakage.py</code>                                     |
| FR-13 | M3     | Margin-aware Elo with season carry-over and history                          | P0   | <code>features/elo.py</code> , <code>models/elo_tuning.py</code>                                                | <code>test_features.py</code>                                    |
| FR-14 | M3     | $\geq 4$ model families and $\geq 4$ baselines under walk-forward validation | P0   | <code>models/train.py</code> , <code>baselines.py</code> , <code>splits.py</code>                               | <code>test_models.py</code>                                      |
| FR-15 | M3     | Calibration and evaluation report with CIs and a reliability diagram         | P0   | <code>models/evaluate.py</code> , <code>calibrate.py</code>                                                     | <code>test_models.py</code>                                      |
| FR-16 | M3     | Predict via CLI, API and UI; two tiers; explanation                          | P0   | <code>models/predict.py</code> , <code>explain.py</code> , <code>services/prediction_service.py</code>          | <code>test_experiment.py</code> , <code>test_services.py</code>  |
| FR-17 | M3     | First-innings score regressor vs baselines                                   | P1   | <code>models/score_regressor.py</code>                                                                          | <code>test_experiment.py</code>                                  |
| FR-18 | M3     | Model registry with SHA-256, params, data hash and commit                    | P0   | <code>models/registry.py</code>                                                                                 | <code>test_models.py</code>                                      |
| FR-19 | M4     | What-if: toss, venue, opponent $\rightarrow \Delta$ probability              | P1   | <code>simulation/what_if.py</code> , <code>services/scenario_service.py</code>                                  | <code>test_services.py</code>                                    |
| FR-20 | M4     | Monte Carlo season simulation                                                | P2   | <code>simulation/season_monte_carlo.py</code>                                                                   | <code>test_services.py</code>                                    |
| FR-21 | M5     | Structured logging, prediction log, PSI drift monitor                        | P0   | <code>logging_setup.py</code> , <code>db</code> , <code>prediction_log</code> , <code>reporting/drift.py</code> | <code>test_core.py</code> , <code>test_figures_logging.py</code> |
| FR-22 | M5     | Generated model card and report assets                                       | P0   | <code>reporting/model_card.py</code> , <code>figures.py</code> , <code>assets.py</code>                         | <code>test_experiment.py</code>                                  |

## 4.1 Input / output structure of each module

Table 4: Module inputs and outputs

| Module              | Input                                                                                                                      | Output                                                                                                                                       |
|---------------------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| M1 Data engineering | <code>data/raw/ipl_matches.csv</code> (+ appended uploads), <code>configs/*.yaml</code> , <code>data/external/*.csv</code> | <code>matches.parquet</code> , <code>match_players.parquet</code> , SQLite DB, <code>quarantine.csv</code> , quality report, data dictionary |
| M2 Analytics        | Canonical match table, filter values                                                                                       | DataFrames, Plotly charts, <code>reports/analytics.json</code> , <code>docs/analytics_findings.md</code> , safe CSV                          |



| Module             | Input                                                     | Output                                                                                                                     |
|--------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| M3 Prediction      | Canonical matches + squads; config (seeds, grids, splits) | <code>reports/metrics.json</code> , registered models, figures, model card; at serving time: probability, drivers, log row |
| M4 Scenarios       | A fixture (teams, venue, date, optional toss)             | What-if table ( $\Delta$ probability), title/top-4 odds                                                                    |
| M5 Ops & reporting | Pipeline events, predictions, feature frame               | Log file, <code>prediction_log</code> table, PSI table, NFR evidence, this report                                          |

## 4.2 User workflow

The Home page shows key figures and the current data and model status. From there, users can follow three paths (diagram D3, Section 7.2):

- **Explore:** filter, inspect and export.
- **Predict:** select two teams and a venue, optionally enter the toss result, and click Predict. Defaults keep the usual workflow within three clicks.
- **Maintain:** upload, dry run, then apply.

## 5. Non-functional Requirements

We defined eight non-functional requirements, each with a measurable target and a way to check it. The results are recorded in docs/nfr\_verification.md and summarised below.

Table 5: Non-functional requirements: targets and verification method

| ID     | Category                     | Requirement (target)                                                                                                                         | Verified by                                                                                                     |
|--------|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| NFR-01 | Performance                  | Full pipeline $\leq 90$ s; single warm prediction $\leq 200$ ms; dashboard page $\leq 2$ s                                                   | <code>creaseiq benchmark</code> $\rightarrow$ <code>reports/perf.json</code> ; <code>test_performance.py</code> |
| NFR-02 | Reliability & error handling | Typed exception hierarchy; idempotent pipeline; friendly errors with no stack traces                                                         | Unit, CLI and AppTest tests with bad inputs                                                                     |
| NFR-03 | Security                     | Allow-list validation; parameterised SQL only; hash-verified model loading; CSV-only uploads $\leq 5$ MB; CSV-injection escaping; no secrets | <code>bandit</code> , <code>pip-audit</code> , <code>test_security.py</code>                                    |
| NFR-04 | Usability                    | $\leq 3$ clicks to a prediction; colour-blind-safe palettes; tooltips; empty states; disclaimer                                              | AppTest, screenshots                                                                                            |
| NFR-05 | Maintainability              | Type hints, docstrings, ruff and mypy clean, enforced layering, $\geq 85\%$ coverage on core packages                                        | CI, <code>coverage.json</code> , <code>test_architecture.py</code>                                              |
| NFR-06 | Reproducibility              | Seeds, pinned dependencies, config-driven; identical metrics across runs                                                                     | <code>test_experiment.py::test_reproducible_metrics</code>                                                      |
| NFR-07 | Logging & monitoring         | Structured logs with run id and timings; prediction log; PSI drift panel                                                                     | <code>test_figures_logging.py</code> , Model Lab                                                                |
| NFR-08 | Scalability & efficiency     | DB backend by URL; <code>MatchSource</code> interface; $O(n)$ incremental features; RAM $< 1$ GB                                             | <code>perf.json</code> , interface tests                                                                        |

### 5.1 Measured results

Table 6: Measured NFR results (generated from evidence files)

| ID     | Category                     | Measured                                                                                                                                                                                                                                                                                                         | Result |
|--------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| NFR-01 | Performance                  | pipeline ingest $\rightarrow$ DB $\rightarrow$ features $\rightarrow$ train of the registered configuration 6.6 s ( $\leq 90$ ; the one-off model-selection search is reported separately in <code>metrics.json</code> ); prediction 9.3 ms median, p95 10.9 ms ( $\leq 200$ ); slowest page 0.57 s ( $\leq 2$ ) | PASS   |
| NFR-02 | Reliability & error handling | typed exception hierarchy; CLI exit code 1 with a friendly message; UI guard with no stack traces; idempotent DB rebuild (checksum test)                                                                                                                                                                         | PASS   |
| NFR-03 | Security                     | bandit issues: 0; known vulnerable dependencies: 0; parameterised SQL, hash-verified artifacts, upload guards and CSV-injection escaping all tested                                                                                                                                                              | PASS   |
| NFR-04 | Usability                    | prediction in $\leq 3$ clicks (defaults preselected, one Predict button); colour-blind-safe palettes; tooltips, empty states and a disclaimer on every page                                                                                                                                                      | PASS   |

| ID     | Category                          | Measured                                                                                                                                    | Result |
|--------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------|
| NFR-05 | Maintainability                   | coverage total 94.6%, core minimum 98.7% (data 99%, features 100%, models 99%, analytics 100%); ruff, mypy and the layer contract are clean | PASS   |
| NFR-06 | Reproducibility                   | seeded, config-driven, pinned dependencies; two full experiment runs give identical metrics                                                 | PASS   |
| NFR-07 | Logging & monitoring              | key=value logs with a run_id and per-stage durations (console plus rotating file); prediction_log table; PSI drift report                   | PASS   |
| NFR-08 | Scalability & resource efficiency | peak Python memory 40 MB (< 1024); DB backend chosen by URL; MatchSource protocol; incremental O(n) features; Parquet cache                 | PASS   |

Measured on Windows-11-10.0.26200-SP0 (16 logical CPUs, Python 3.12.10).

Pipeline stage times (seconds):

| Stage                                 | Time |
|---------------------------------------|------|
| Data                                  | 2.33 |
| Database                              | 2.70 |
| Features                              | 1.47 |
| Training the registered configuration | 0.09 |

NFR-01 covers the operational pipeline, which trains the registered configuration. The one-off model-selection experiment is not part of it. That experiment (the Elo grid, five model families and their grids, 13 walk-forward folds, two tiers, calibration, ablations and bootstrap tests) took 106.5 s in the recorded `creaseiq train run`.

The application was checked on Windows 11, including installation and execution from a clean clone. The GitHub Actions workflow is set up for Ubuntu, Windows and macOS with Python 3.12 and 3.13. It will run on each push after the repository is published.

## 6. System Architecture

CreaseIQ separates the interface, project logic and data handling into layers. The Streamlit dashboard and Typer CLI call shared services. These services coordinate analytics, features, models, simulations, reports and charts. The data-access layer handles input, validation, cleaning and database operations. Shared configuration, logging and error utilities support the layers, and dependencies point downward.

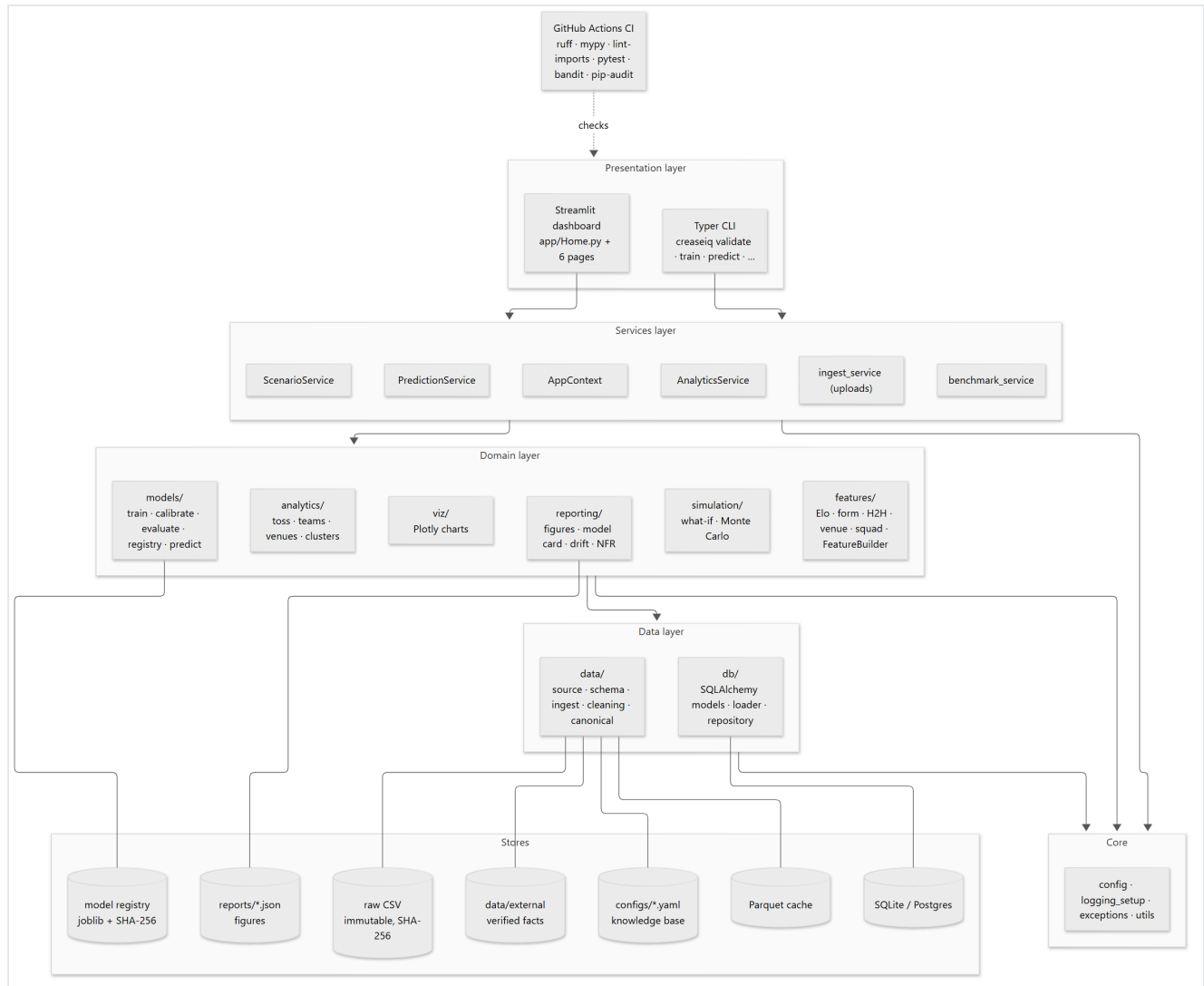


Figure 1: System architecture (D1): layers, stores and CI

Table 7: Layers and responsibilities

| Layer          | Packages                                                | Responsibility                                                         |
|----------------|---------------------------------------------------------|------------------------------------------------------------------------|
| 4 Presentation | app, cli                                                | Widgets and commands, rendering, friendly errors                       |
| 3 Services     | services                                                | Use cases: validate, orchestrate, log, time                            |
| 2 Domain       | analytics, features, models, simulation, reporting, viz | Statistics, feature engineering, machine learning, simulation, figures |
| 1 Data access  | data, db                                                | Sources, schema, cleaning, canonical maps, ORM, repositories           |
| 0 Core         | config, logging_setup, exceptions, utils                | Cross-cutting concerns                                                 |

We check the layer boundaries with import-linter in CI and an AST test in tests/unit/test\_architecture.py. The test rejects imports from a lower layer into a higher one. This keeps the design consistent:

- The dashboard and the CLI share one implementation of every use case.
- Domain code is testable without the UI.
- A new front end (for example a REST API) would only add a layer-4 module.

#### Data stores:

- **Raw CSV.** Immutable; its SHA-256 is checked on every run.
- **Verified external facts.** Super-over winners and the D/L list, with their sources.
- **YAML knowledge base.** Franchise lineage, venues and home grounds.
- **Parquet cache.**
- **SQLite database.** Switchable to PostgreSQL by URL.
- **Hash-verified model registry.**
- **Generated reports and figures.**

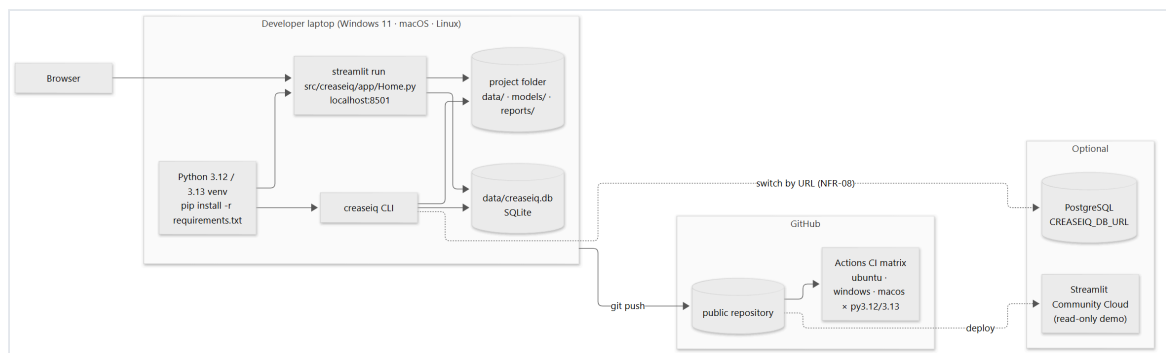


Figure 2: Deployment and environment (D10)

## 6.1 Technology stack

Table 8: Technology choices

| Concern    | Choice                                                           | Reason                                                       |
|------------|------------------------------------------------------------------|--------------------------------------------------------------|
| Language   | Python 3.12 / 3.13                                               | Current numpy and scipy require $\geq 3.12$ (ADR-002)        |
| Data       | pandas 3, NumPy, pyarrow                                         | Standard; columnar cache                                     |
| Validation | pandera                                                          | Declarative schemas with lazy failure collection             |
| Statistics | SciPy, statsmodels                                               | Exact tests; cross-checked results                           |
| ML         | scikit-learn                                                     | All model families without native OpenMP dependencies        |
| Storage    | SQLAlchemy 2 + SQLite                                            | Zero administration; Postgres available by URL               |
| UI         | Streamlit + Plotly                                               | Python-only interactive dashboard, testable with AppTest     |
| CLI        | Typer + Rich                                                     | Typed commands with <code>--help</code>                      |
| Quality    | pytest, Hypothesis, ruff, mypy, import-linter, bandit, pip-audit | Automated gates in CI                                        |
| Docs       | Mermaid, matplotlib, Playwright/Chromium                         | Diagrams as code; the same renderer on all operating systems |

## 7. Design Diagrams

---

All diagrams are written as Mermaid sources in `docs/diagrams/` and drawn from the actual code: class names, table columns and import rules match the source files.

### 7.1 Use case diagram

Three actors use the system:

- **Analyst / student:** explore, analyse and export.
- **Cricket fan:** predict, what-if and simulate.
- **Maintainer:** ingest, append, rebuild, train and benchmark.

Each use case carries its FR identifiers. Mermaid has no native use-case notation, so actors are drawn as circles and use cases as rounded shapes inside the system boundary.

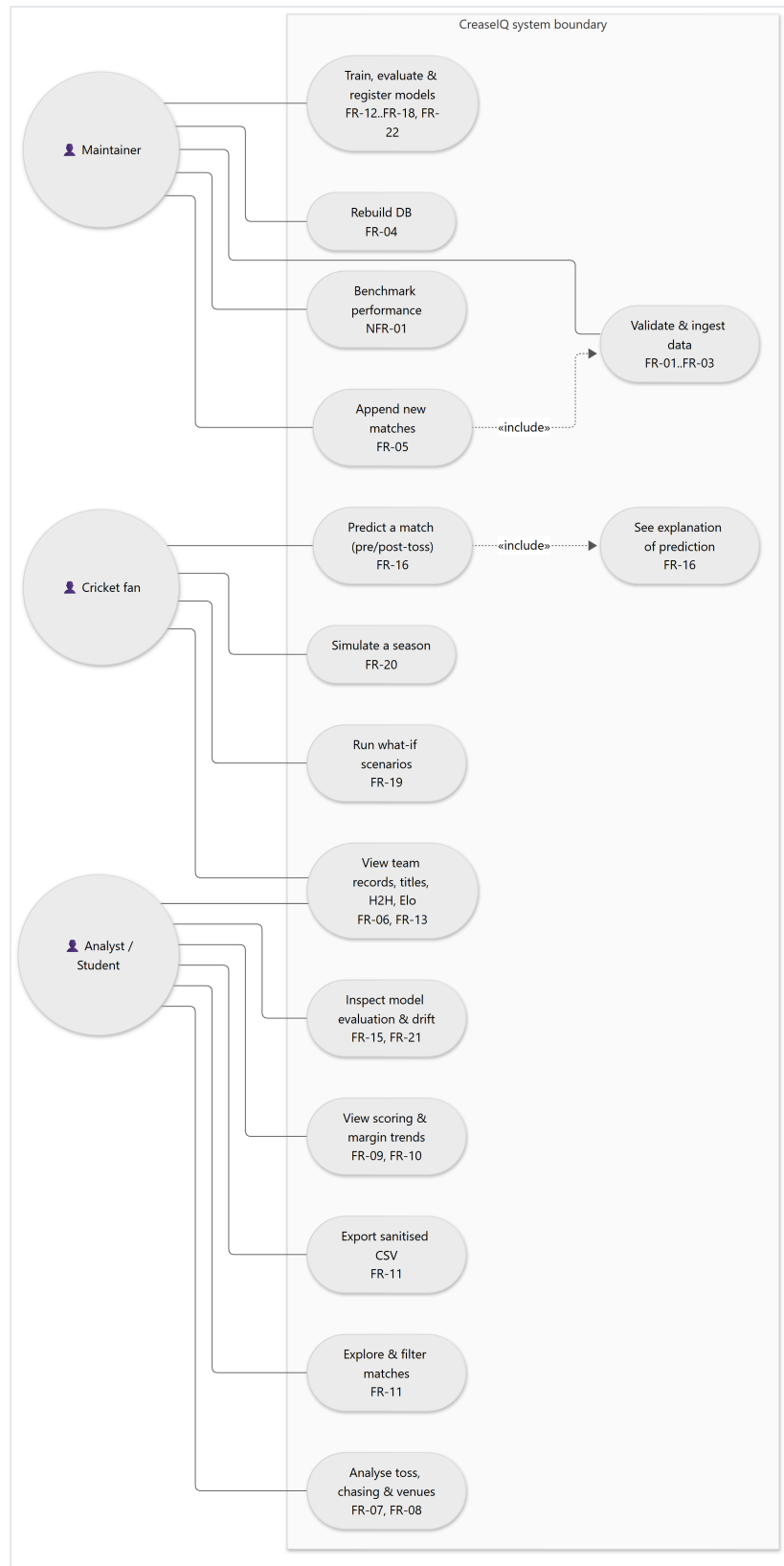


Figure 3: Use case diagram (D4)

## 7.2 Workflow diagrams

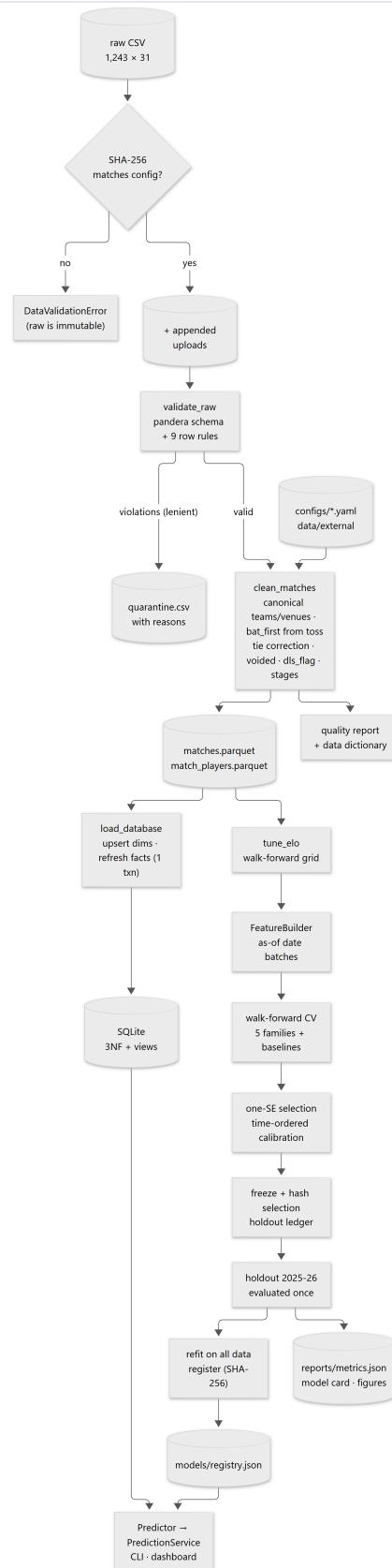


Figure 4: Process flow: the data and modelling pipeline run by `creaseiq all` (D2)



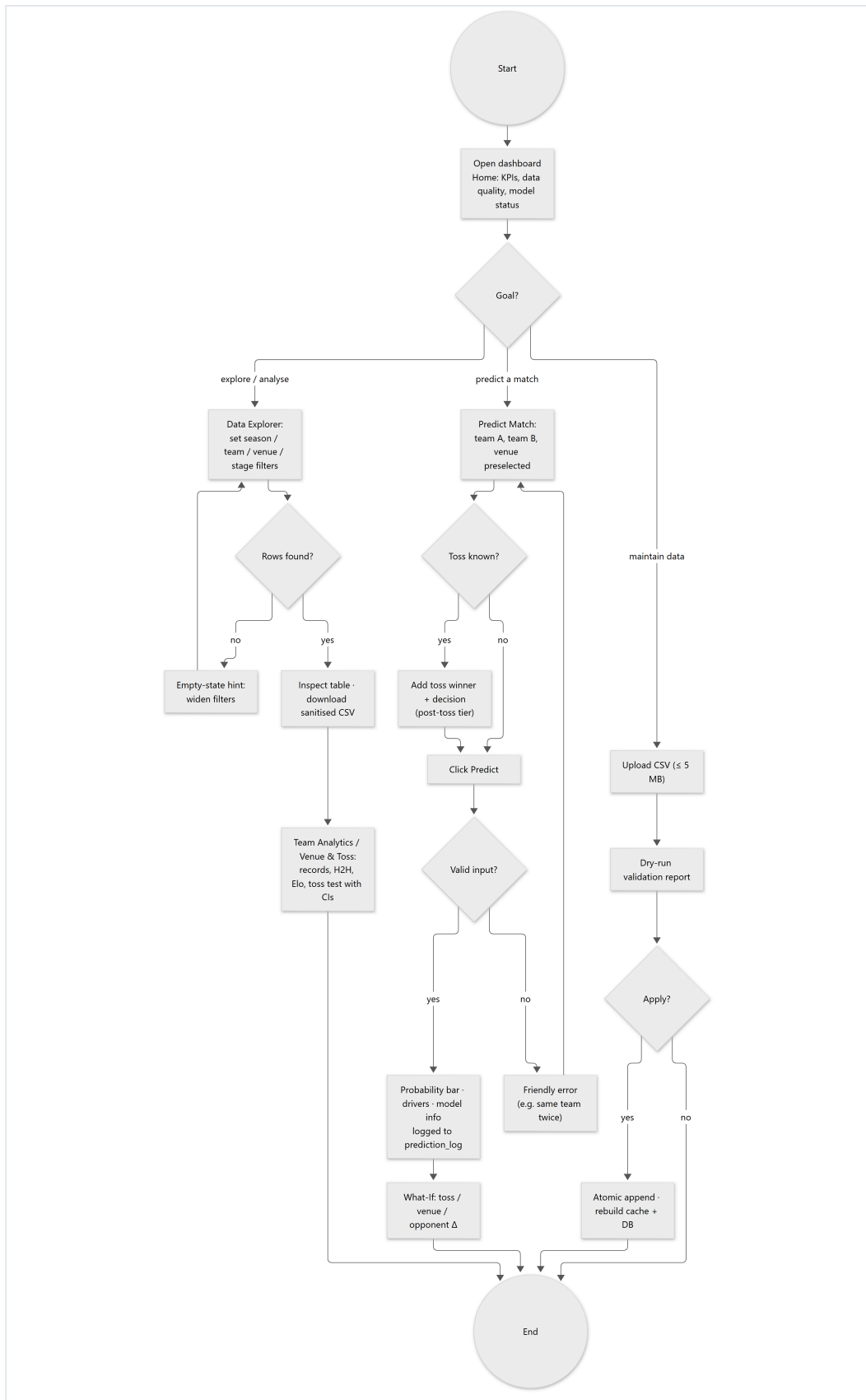


Figure 5: User workflow (activity diagram): analyst, predictor and maintainer paths (D3)

## 7.3 Sequence diagrams

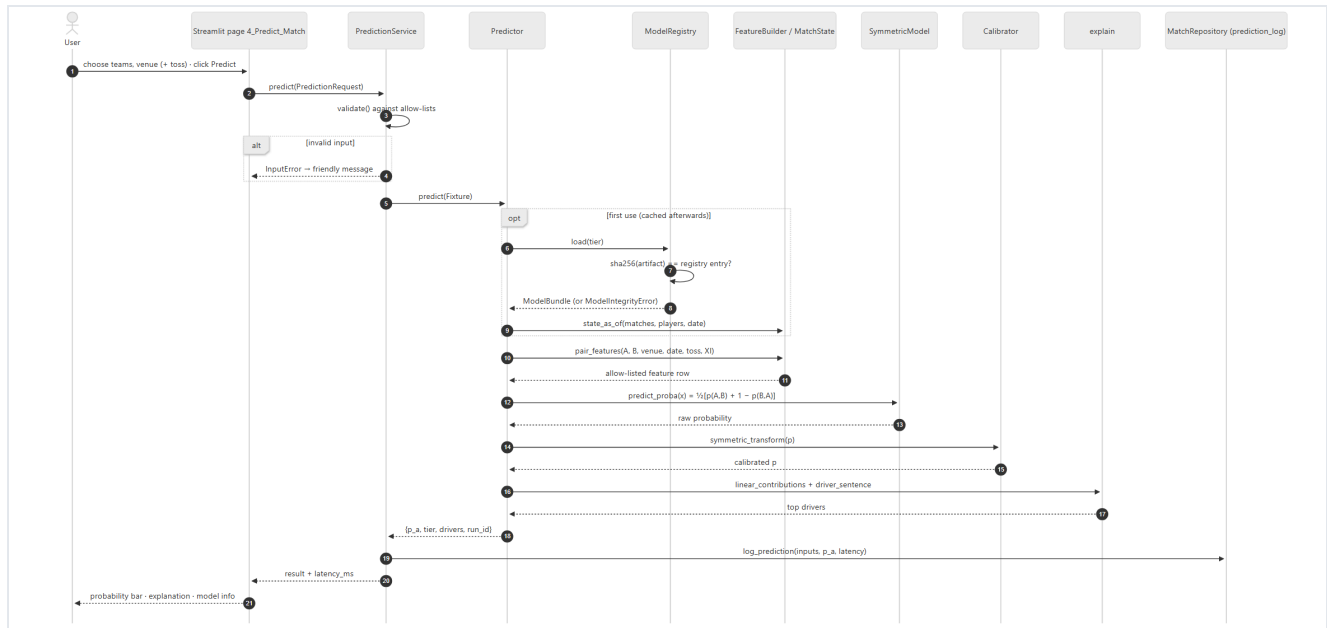


Figure 6: Sequence: serving one prediction, from validation to the prediction log (D5)

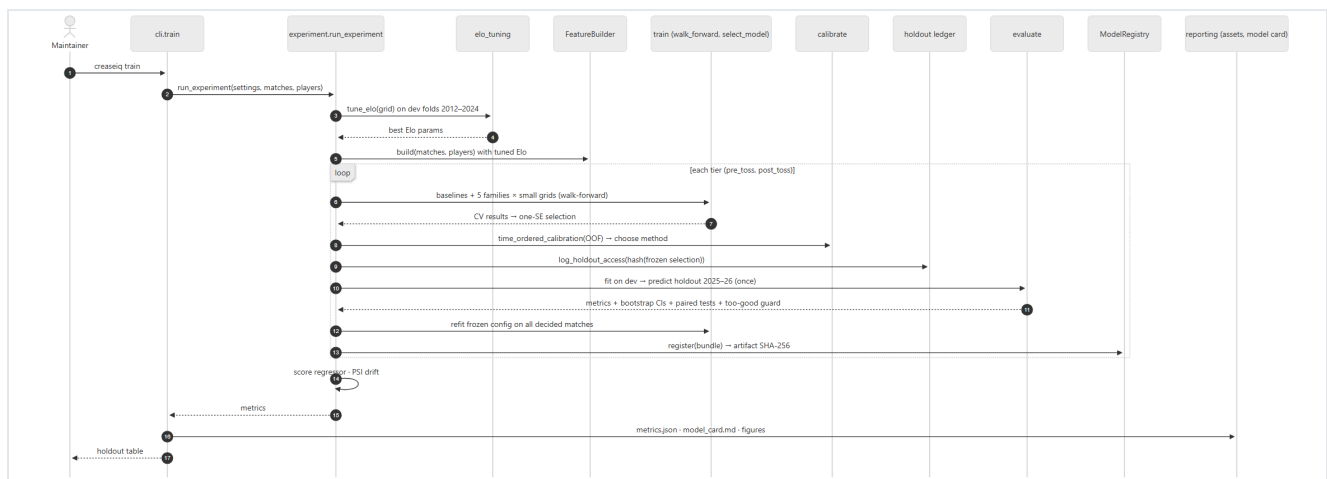


Figure 7: Sequence: training, selection, one-shot holdout evaluation and registration (D6)

## 7.4 Class and component diagrams

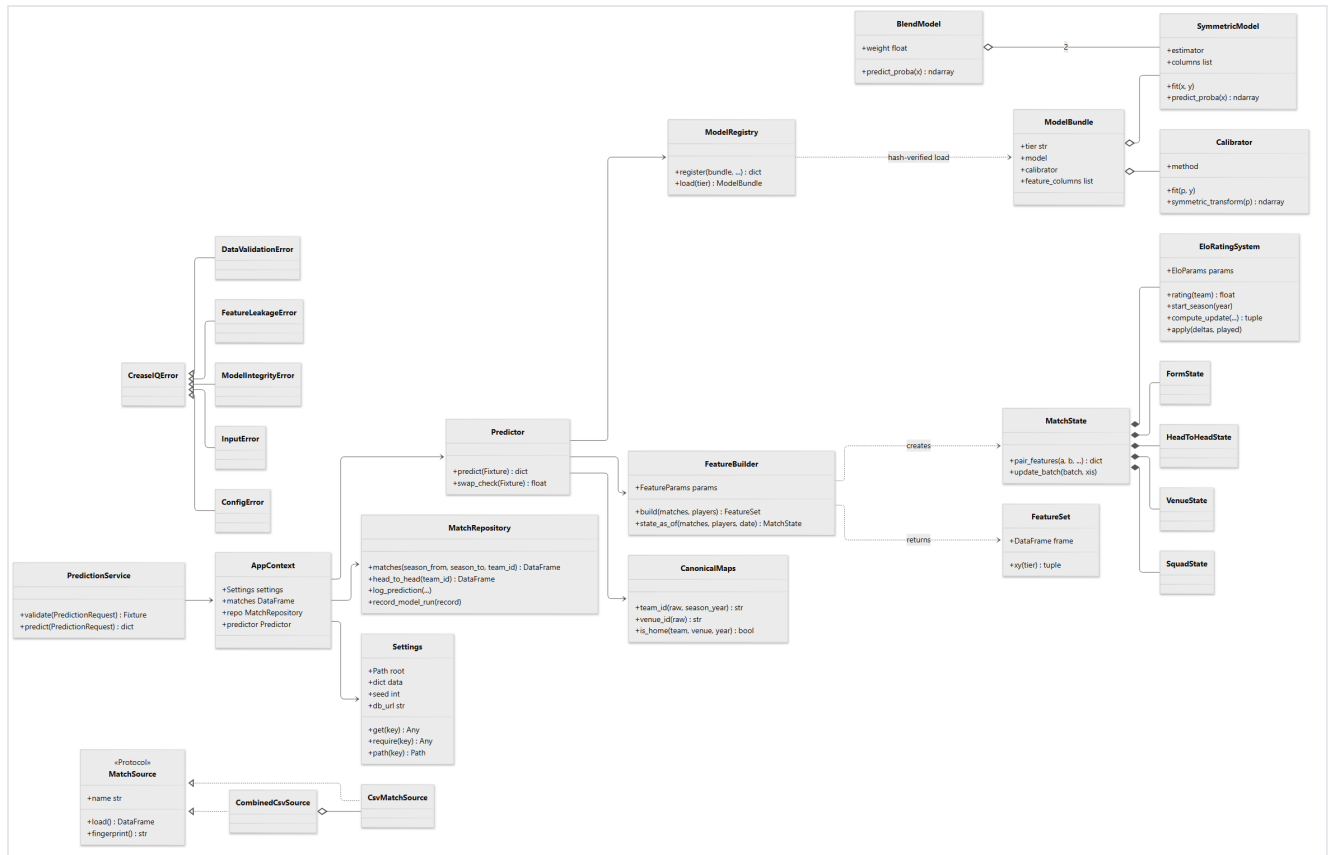


Figure 8: Class diagram of the key classes (D7)

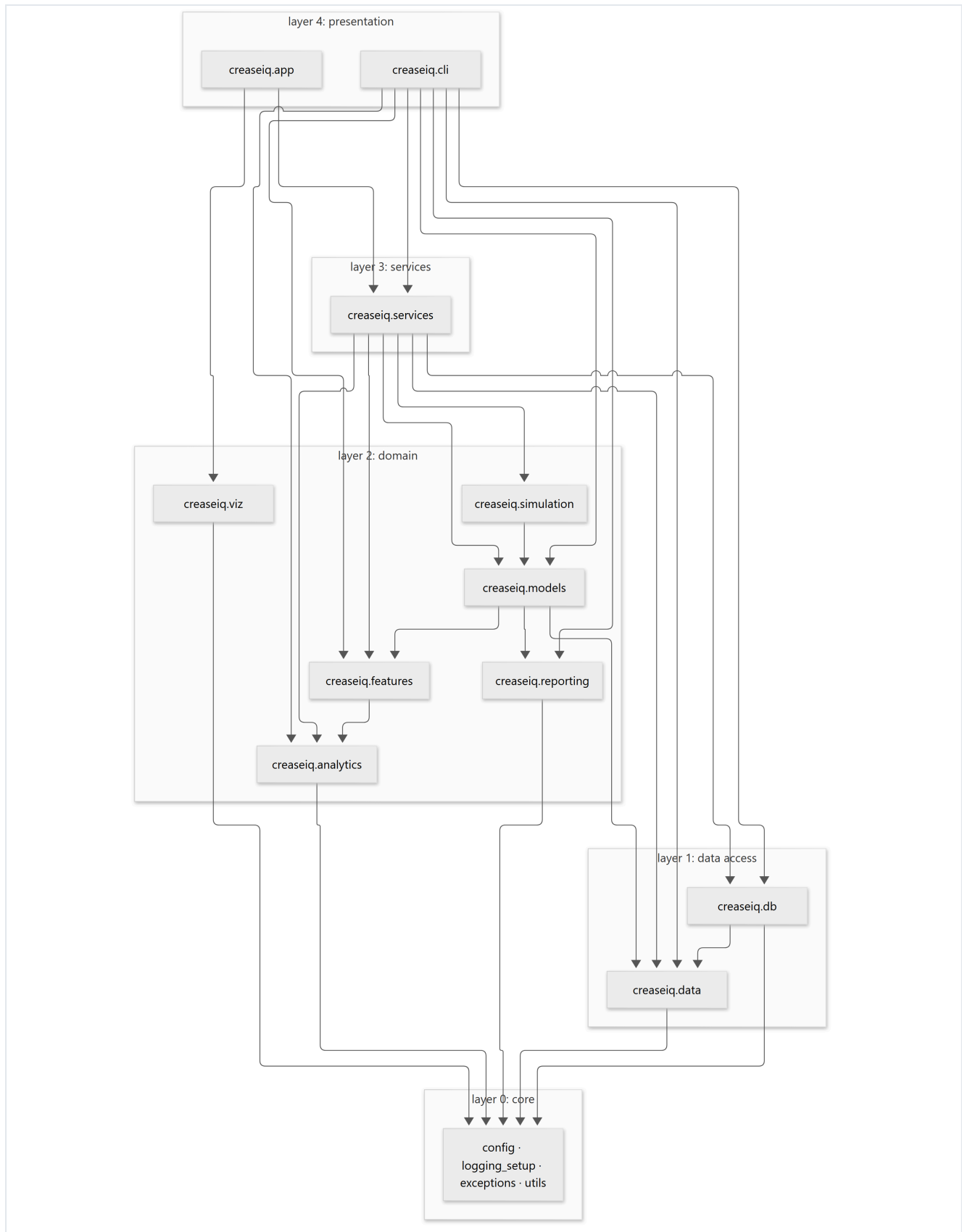


Figure 9: Component / package diagram; arrows are the only allowed imports (D8)

## 7.5 ER diagram and schema

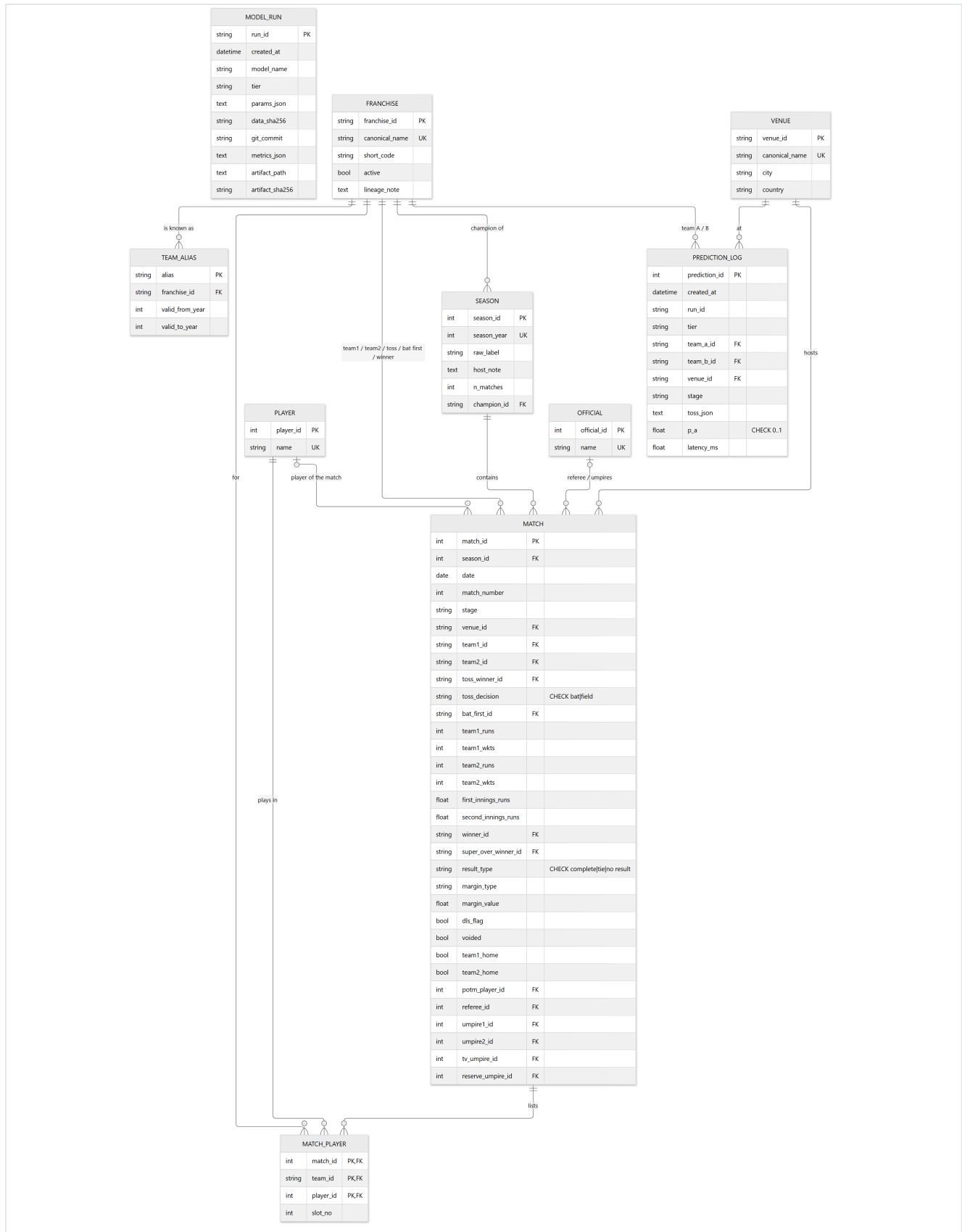


Figure 10: Entity-relationship diagram generated from the ORM models (D9)

**Schema summary.** The schema has ten tables and two views:

- **Dimension tables:** franchise, team\_alias, venue, season, player, official.

- **Fact tables:** `match` and the associative `match_player`.
- **Operational tables:** `model_run` and `prediction_log`.
- **Views:** `v_team_season_summary` and `v_head_to_head`.

**Constraints.** The database enforces:

- a natural key, `UNIQUE(date, team1_id, team2_id)`;
- `CHECK` s on the toss decision, result type, distinct teams, "winner present iff complete" and probabilities in `[0, 1]`;
- foreign keys, with `PRAGMA foreign_keys=ON`.

The full DDL is generated from the models into `docs/schema.sql`.

**Normalisation (3NF).** Every non-key attribute depends on the key, the whole key and nothing but the key:

- Franchise names live only in `franchise`, and aliases (with validity years) in `team_alias`.
- Venue attributes live only in `venue`.
- Player and official names live in their own tables, referenced by surrogate ids.
- Squads form a many-to-many relation ( `match_player`, composite key `match_id, team_id, player_id`), not a repeating group.
- `match` holds only foreign keys and attributes of the match itself.

The one deliberate redundancy is `season.champion_id`, which can be derived from the season's final. It is kept as a documented, test-verified convenience.

## 7.6 Model lifecycle

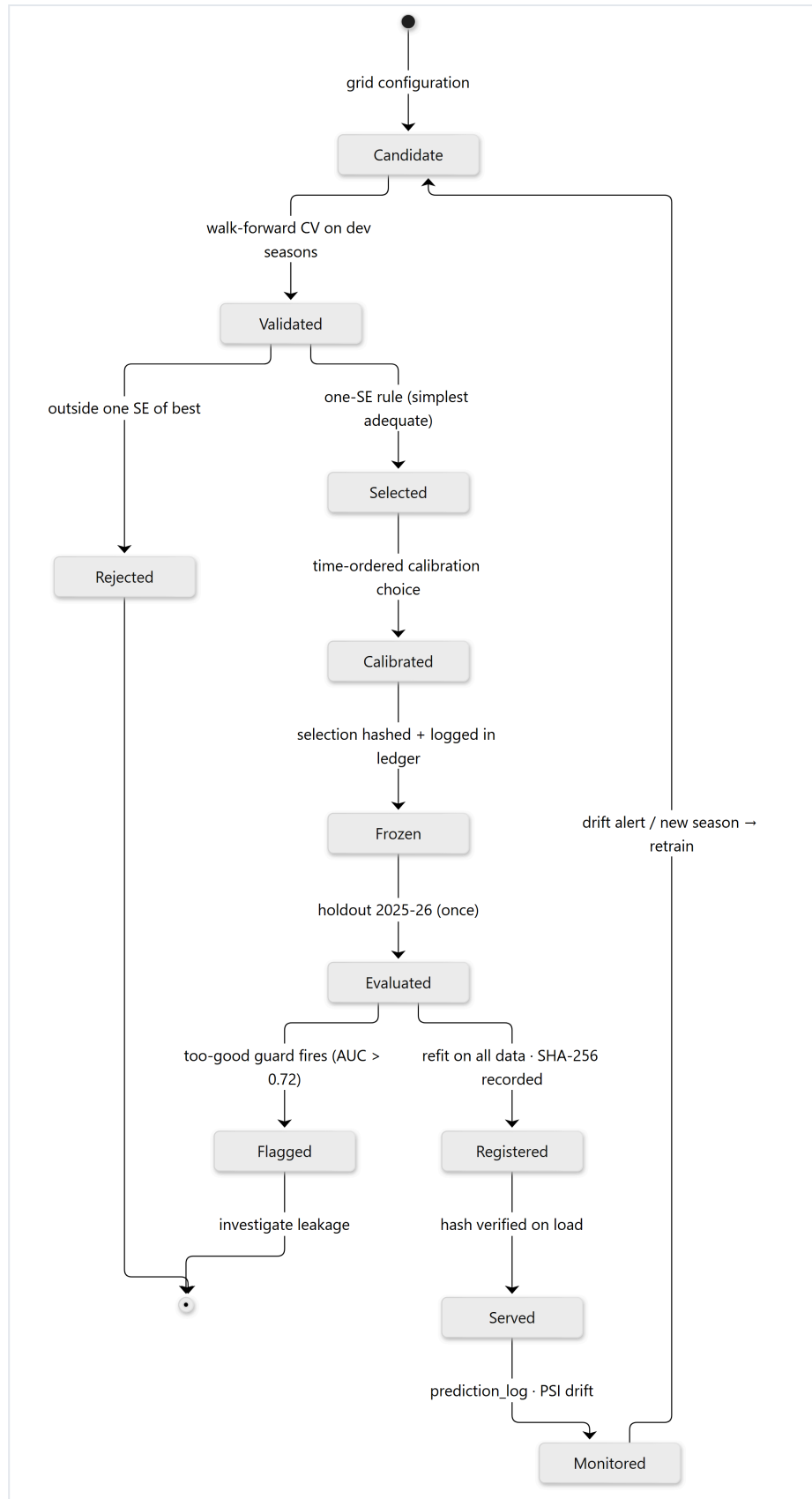


Figure 11: Model lifecycle state diagram (D12)

## 8. Design Decisions & Rationale

We recorded the main design choices in Architecture Decision Records (ADRs) under docs/decisions/. Each record explains the problem, the alternatives we considered, our choice and its consequences.

Table 9: Architecture decision records

| ADR                                 | Decision                                                                                      | Why                                                                                                          |
|-------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| 001 Course relevance                | ML core, with AI fundamentals mapped explicitly; adds k-means venue profiling                 | Connects the project to the syllabus topics of agents, search, knowledge representation and machine learning |
| 002 Stack                           | Python $\geq 3.12$ ; scikit-learn only; Playwright for PDFs                                   | numpy 2.5 and scipy 1.18 dropped 3.11; avoids native OpenMP and GTK installs on Windows                      |
| 003 Franchise continuity            | Renames share one id; new owner or contract means a new franchise                             | Legally correct; fixes title counts and head-to-head; sensitivity documented                                 |
| 004 Data corrections                | Correct ties with regulation scores, flag voided and D/L rows; never quarantine them          | Keeps 16 real matches; the raw values stay available for audit                                               |
| 005 Orientation & symmetry          | Hash-seeded coin flip per match; declared antisymmetric features; symmetrised predictions     | Raw <code>team1</code> order leaks batting order from 2018                                                   |
| 006 Selection, calibration, holdout | Walk-forward CV; one-SE rule; time-ordered calibration; hashed one-shot holdout with a ledger | Prevents overfitting noise and silent re-use of the holdout                                                  |

### 8.1 The orientation trap (ADR-005)

The classifier needs a consistent way to label teams A and B. Using raw `team1` for team A would introduce a problem: from 2018 onward, `team1` always bats first. Batting first wins only 45.3% of decided matches, so this ordering could reveal batting order even to a model intended for use before the toss.

CreaseIQ handles this in four steps:

1. Each match is oriented by a SHA-256-seeded coin flip on its own date and teams, so deleting other matches cannot change it.
2. Every feature is declared *antisymmetric* (a difference or  $\pm 1$  flag) or *symmetric context* (venue, stage, era).
3. The logistic model has no intercept and uses only antisymmetric features. That makes it exactly symmetric:  $p(A,B) = 1 - p(B,A)$ .
4. Tree models are symmetrised at prediction time, and a test checks invariance to  $1e-9$ .

### 8.2 Leakage-safe features ("as-of")

Features are built one date at a time. For every match on that date, the builder uses only the state from earlier dates. It updates the state after all matches on the date are processed, so double-headers cannot reveal information about each other. Five tests check this: future deletion, future mutation, same-day isolation, label shuffling and the column allow-list (Section 11).

### 8.3 Why walk-forward, the one-SE rule and log-loss

Random k-fold would train on future seasons. The IPL is non-stationary (auctions, the Impact Player rule, neutral seasons), and Cerqueira et al. recommend time-ordered out-of-sample evaluation for such series



[11]. CreaseIQ therefore validates each development season 2012-2024 with a model trained on all earlier seasons.

With about 1,000 development matches, small differences between models can be unreliable. We used the one-standard-error rule to choose the simplest model close to the best average result. Log-loss is the main measure because it considers how confident a prediction is. We also checked calibration, rather than relying on accuracy alone [9].

## 8.4 Calibration choice

We compared no calibration, Platt scaling (sigmoid) and isotonic regression. For each season, calibration used only out-of-fold predictions from earlier seasons. Isotonic regression can overfit with fewer than about 1,000 calibration samples [12], [13]. In this experiment, leaving probabilities uncalibrated gave the best results:

| Tier      | none   | sigmoid | isotonic | Chosen |
|-----------|--------|---------|----------|--------|
| post_toss | 0.6852 | 0.6861  | 0.7209   | none   |
| pre_toss  | 0.6914 | 0.6918  | 0.7068   | none   |

## 8.5 One-shot holdout with a ledger

We reserved 2025-26 for holdout evaluation. Before evaluation, the model, parameters, calibration, Elo settings, features and data hash were frozen into a selection ID. This ID is recorded in

`reports/holdout_ledger.json`, which counts every evaluation.

Running the same frozen selection again gives the same results. A different selection would be flagged as holdout reuse. ADR-006 also records that the ledger was reset once after an exploratory run of the same selection.

## 8.6 Other decisions

- **SQLite vs a server database:** zero administration for a single-user app; PostgreSQL remains one URL away (NFR-08).
- **Streamlit:** a Python-only UI, with `AppTest` for automated UI tests.
- **Correct known errors:** when an anomaly has a verified explanation, we correct it. Unrecognised values produce a clear error instead of being guessed.
- **Toss analysis:** winning a random toss can be studied as a causal question. Choosing to bat or field is a captain's decision, so comparisons based on that choice are reported as associations [2].
- **Small samples:** venue and team-at-venue rates are adjusted towards league averages, so a few matches do not create misleadingly strong conclusions.

## 9. Implementation Details

### 9.1 Dataset description

The input is a Cricsheet-derived IPL match file [1]:

- **Size:** 1,243 rows × 31 columns, one row per match, from 2008-04-18 to 2026-05-31.
- **Integrity:** SHA-256 `db81a92c740ef6d9...`, stored immutably.
- **Columns:** event, season label, match number, date, city, venue, the two teams, toss winner and decision, runs and wickets for each team, winner, result type, margins, player of the match, five officials, six constant columns (T20, 20 overs, 6 balls, male, club), and the two squad lists.
- **Squads:** 11 players each through 2022, and 12 from 2023, when the Impact Player substitute is listed.

Table 10: Outcome composition

| Matches | Decided | Ties | No result | Voided | Distinct players | Toss decision field / bat |
|---------|---------|------|-----------|--------|------------------|---------------------------|
| 1,243   | 1,218   | 16   | 9         | 1      | 811              | 825 / 418                 |

Table 11: Seasons and derived champions (champions verified against published results for all seasons)

| Season | Label   | Matches | Decided | Ties | No result | Champion        |
|--------|---------|---------|---------|------|-----------|-----------------|
| 2008   | 2007/08 | 58      | 58      | 0    | 0         | rr              |
| 2009   | 2009    | 57      | 56      | 1    | 0         | deccan_chargers |
| 2010   | 2009/10 | 60      | 59      | 1    | 0         | csk             |
| 2011   | 2011    | 73      | 72      | 0    | 1         | csk             |
| 2012   | 2012    | 74      | 74      | 0    | 0         | kkr             |
| 2013   | 2013    | 76      | 74      | 2    | 0         | mi              |
| 2014   | 2014    | 60      | 59      | 1    | 0         | kkr             |
| 2015   | 2015    | 59      | 56      | 1    | 2         | mi              |
| 2016   | 2016    | 60      | 60      | 0    | 0         | srh             |
| 2017   | 2017    | 59      | 58      | 1    | 0         | mi              |
| 2018   | 2018    | 60      | 60      | 0    | 0         | csk             |
| 2019   | 2019    | 60      | 57      | 2    | 1         | mi              |
| 2020   | 2020/21 | 60      | 56      | 4    | 0         | mi              |
| 2021   | 2021    | 60      | 59      | 1    | 0         | csk             |
| 2022   | 2022    | 74      | 74      | 0    | 0         | gt              |
| 2023   | 2023    | 74      | 73      | 0    | 1         | csk             |
| 2024   | 2024    | 71      | 71      | 0    | 0         | kkr             |
| 2025   | 2025    | 74      | 70      | 1    | 3         | rcb             |
| 2026   | 2026    | 74      | 72      | 1    | 1         | rcb             |

### 9.2 Validation and cleaning

**Validation** has two layers:

1. A pandera schema checks types, value ranges, allowed values and the six constant columns.
2. Nine vectorised row rules check cross-column logic:

- the teams are distinct;
- the toss winner and the winner both took part in the match;
- a winner is present exactly when the result is "complete";
- exactly one margin is recorded;
- there are no margins on ties;
- at most 10 wickets except on ties;
- the date is valid;
- each squad lists 11-13 players.

In lenient mode, failing rows go to `quarantine.csv` with named reasons. Strict mode raises with the row indices. The real file passes strict validation with **0 quarantined rows**.

**Cleaning** records every change it makes:

*Table 12: Fixes applied by the cleaning pipeline (from the data-quality report)*

| Fix                                      | Rows  |
|------------------------------------------|-------|
| city_corrected                           | 170   |
| city_unknown_imputed                     | 51    |
| dls_flag_external                        | 16    |
| dls_flag_heuristic                       | 14    |
| dls_flag_total                           | 19    |
| no_result_scores_excluded                | 9     |
| player_lists_parsed                      | 2,486 |
| season_label_mapped                      | 178   |
| stage_derived_playoff                    | 74    |
| team_alias_canonicalised                 | 651   |
| tie_innings_corrected_super_over_removed | 16    |
| venue_alias_canonicalised                | 862   |
| voided_flagged                           | 1     |

**Derived columns** (all documented in `docs/data_dictionary.md`):

- `season_year` and batting order from the toss.
- Regulation first- and second-innings runs.
- An era-specific playoff `stage`, `season_champion` and `impact_era`.
- Home flags from a researched home-ground knowledge base, with neutral seasons covering 2009, 2020, 2021 and 2022.
- `dls_flag`, `voided` and `scores_usable`.

## 9.3 Feature engineering

Every feature is computed *as of the day before* the match (Section 8.2) and oriented A – B. The groups are:

*Table 13: Feature groups (tier B adds the toss and squad groups)*

| Group | Features                                       | Notes                                                                     |
|-------|------------------------------------------------|---------------------------------------------------------------------------|
| Elo   | <code>elo_diff</code> , <code>home_diff</code> | Margin-aware Elo, tuned; home bonus only when exactly one side is at home |

| Group             | Features                                                                                                    | Notes                                                    |
|-------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Form              | last-5 and last-10 smoothed win rate, runs scored and conceded (last 10), season win % and games, rest days | Smoothed toward 0.5 (prior strength 2)                   |
| Head-to-head      | all-time and last-5-seasons edge, meetings                                                                  | Edge = (wins_A - wins_B)/(n + 2)                         |
| Venue             | team-at-venue edge, shrunk venue scoring level, shrunk bat-first rate                                       | Priors are the league values <i>so far</i>               |
| Context           | playoff flag, Impact Player era                                                                             | Symmetric                                                |
| Toss (post-toss)  | A bats first, A won the toss, batting order × venue, batting order × era                                    | Interactions let linear models use venue and era context |
| Squad (post-toss) | experience of the XI, prior POTM awards, debutants, continuity (Jaccard)                                    | Named XI known at the toss                               |

### Elo engine.

- **Expected score:**  $1/(1 + 10^{-(R_A - R_B + H)/400})$ .
- **Update:**  $K \times \text{margin multiplier} \times (S - E)$ , where the multiplier is  $\ln(\text{units} + 1) \cdot 2.2/(0.001 \cdot \Delta_{\text{winner}} + 2.2)$  [6].
- **Margin units:** runs/10 or wickets/2.
- **D/L results:** the multiplier is 1.
- **Season carry-over:** ratings regress toward 1500 each season.

The Elo parameters were tuned on development folds over 400 combinations. Best: `home_bonus=75, k=5, season_regression=0.1, use_margin=True`, with walk-forward log-loss 0.6900 against 0.6910 for the defaults.

## 9.4 Model selection rationale

With about 1,200 matches and weak signal, the model families are deliberately simple and their search grids small:

Table 14: Walk-forward mean log-loss of the best configuration of each family, with baselines (development seasons)

| Candidate           | Pre-toss        | Post-toss       |
|---------------------|-----------------|-----------------|
| elo_logit           | 0.6900 ± 0.0172 | 0.6900 ± 0.0172 |
| logreg              | 0.6908 ± 0.0085 | 0.6857 ± 0.0126 |
| random_forest       | 0.6924 ± 0.0117 | 0.6906 ± 0.0127 |
| hist_gb             | 0.6949 ± 0.0138 | 0.6909 ± 0.0195 |
| blend               | 0.6909 ± 0.0097 | 0.6862 ± 0.0138 |
| B0 coin flip        | 0.6931          | 0.6931          |
| B3 Elo probability  | 0.6920          | 0.6920          |
| B1 chase prior      | n/a             | 0.6902          |
| B2 toss winner      | n/a             | 0.6940          |
| B4 venue chase rate | n/a             | 0.6993          |

The families:

- **Elo-logit.** Interpretable, with two features.
- **L2 logistic regression.** No intercept and antisymmetric features only (Section 8.1).
- **Random forest and histogram gradient boosting.** They can use the symmetric context through interactions.

- **Blend.** Of logistic regression and gradient boosting.

Selection used the one-SE rule:

- **Pre-toss: elo\_logit**  $C=1.0$  . The best mean belonged to elo\_logit.
- **Post-toss: logreg**  $C=0.003$  . The best mean belonged to logreg.

The tree ensembles did not beat the linear models. That is expected with  $n \approx 1k$  and weak signal, where variance dominates.

## 9.5 Evaluation methodology

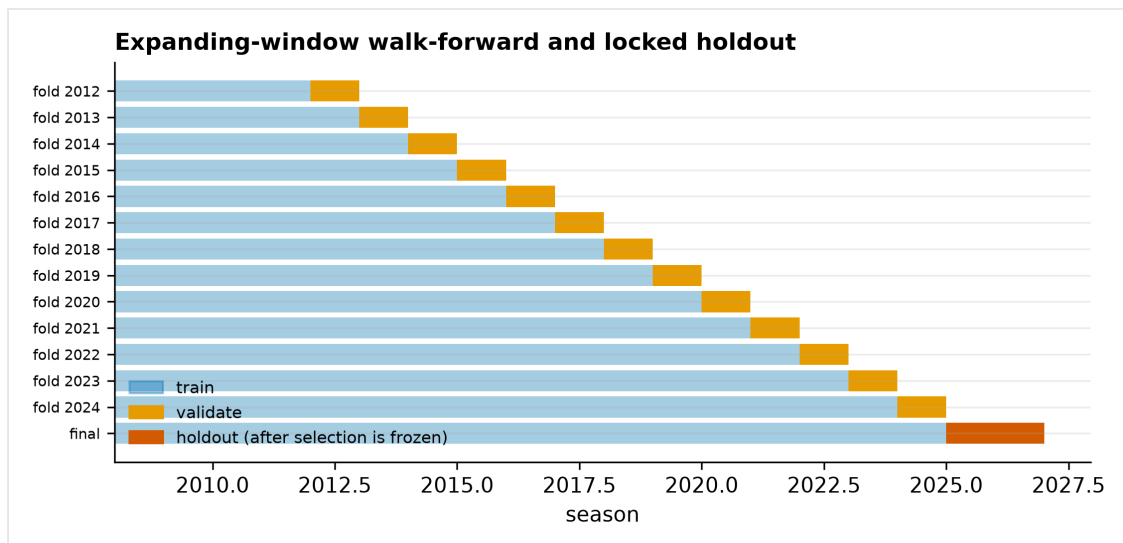


Figure 12: Expanding-window walk-forward folds and the locked holdout (D11)

- **Splits:**
  - Development seasons are  $\leq 2024$ , with folds validating seasons 2012-2024.
  - The holdout is 2025, 2026:  $n = 142$  decided matches, evaluated once per frozen selection.
- **Metrics:**
  - Primary: log-loss.
  - Also: Brier score with its Murphy decomposition, accuracy, ROC-AUC, and ECE on equal-mass bins (descriptive only [14]-[16]).
- **Uncertainty:** a percentile bootstrap (2,000 resamples, seed 42) for every holdout metric.
- **Comparison with baselines:** a paired bootstrap of per-match log-loss differences (10,000 resamples), plus the Diebold-Mariano test with the Harvey-Leybourne-Newbold correction [18].
- **Baselines:** all rates are fitted on the training fold only. The plan's all-data chase rate would have leaked the holdout base rate (plan review finding 6).
- **Guards:**
  - A "too good" check flags holdout AUC above 0.72 or accuracy above 68% as suspected leakage.
  - Ablations add feature groups cumulatively.
  - Performance is broken down by era and stage.

## 9.6 Module walkthrough

Table 15: Packages and their responsibilities

| Package       | Main modules                                                                                                       | Responsibility                                            |
|---------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| data          | source, schema, ingest, canonical, cleaning, quality_report, pipeline                                              | FR-01..03: validated, canonical match table               |
| db            | models, session, loader, repository                                                                                | FR-04: 3NF schema, idempotent load, parameterised queries |
| analytics     | hypothesis_tests, toss_analysis, team_stats, venue_stats, venue_clusters, season_trends, player_stats, summary     | FR-06..10                                                 |
| features      | elo, form, head_to_head, venue_effects, squad, builder                                                             | FR-12, FR-13                                              |
| models        | splits, baselines, train, calibrate, evaluate, elo_tuning, explain, registry, predict, score_regressor, experiment | FR-14..18                                                 |
| simulation    | what_if, season_monte_carlo                                                                                        | FR-19, FR-20                                              |
| services      | context, prediction_service, analytics_service, scenario_service, ingest_service, benchmark_service                | Use cases for the CLI and UI                              |
| reporting     | figures, model_card, drift, nfr, readme, report_builder, assets                                                    | FR-21, FR-22, this report                                 |
| viz, app, cli | Plotly charts; 7 Streamlit pages; 15 CLI commands                                                                  | Presentation                                              |

Besides `version` and `info`, the CLI commands are `validate`, `build-db`, `analyze`, `features`, `train`, `evaluate`, `predict`, `whatif`, `simulate`, `ingest`, `benchmark`, `report` and `all`. Expected errors exit with code 1 and a one-line message.

## 10. Screenshots / Results

### 10.1 Application screenshots

All screenshots are real captures of the running Streamlit app, taken with Playwright ( `scripts/capture_screenshots.py` ).

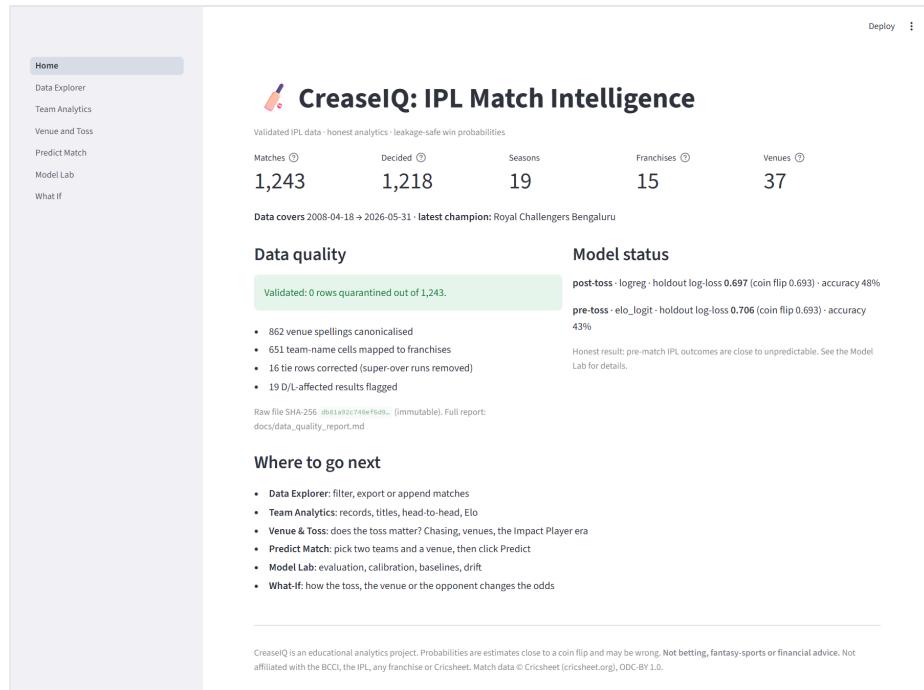
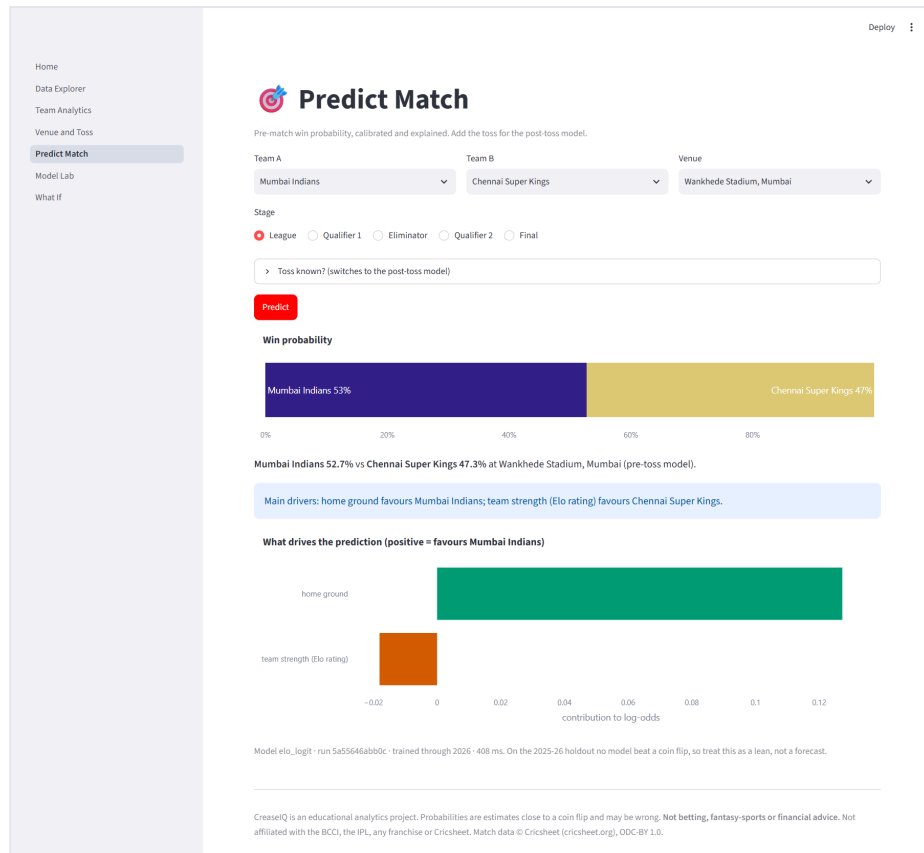


Figure 13: Home: KPIs, data-quality and model status





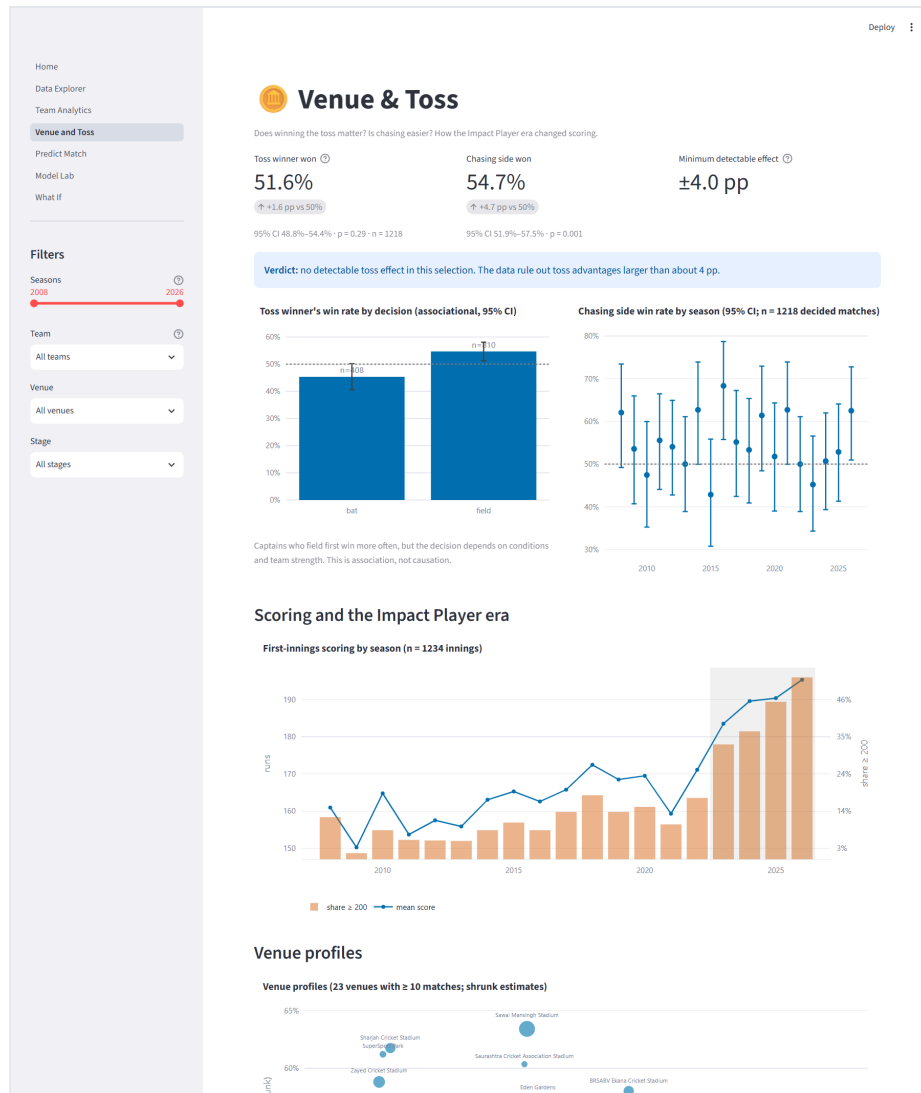


Figure 15: Venue & Toss: causal toss test with CI and minimum detectable effect, chasing by season

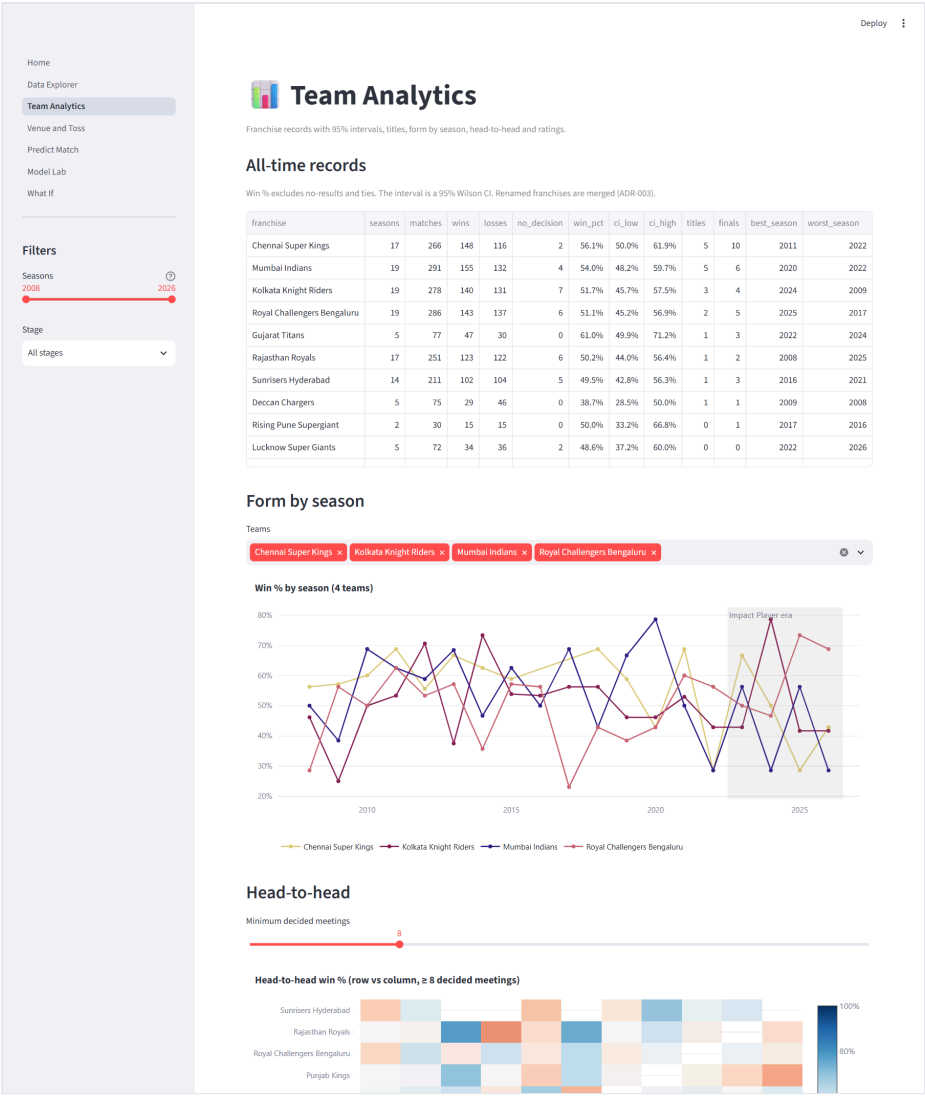
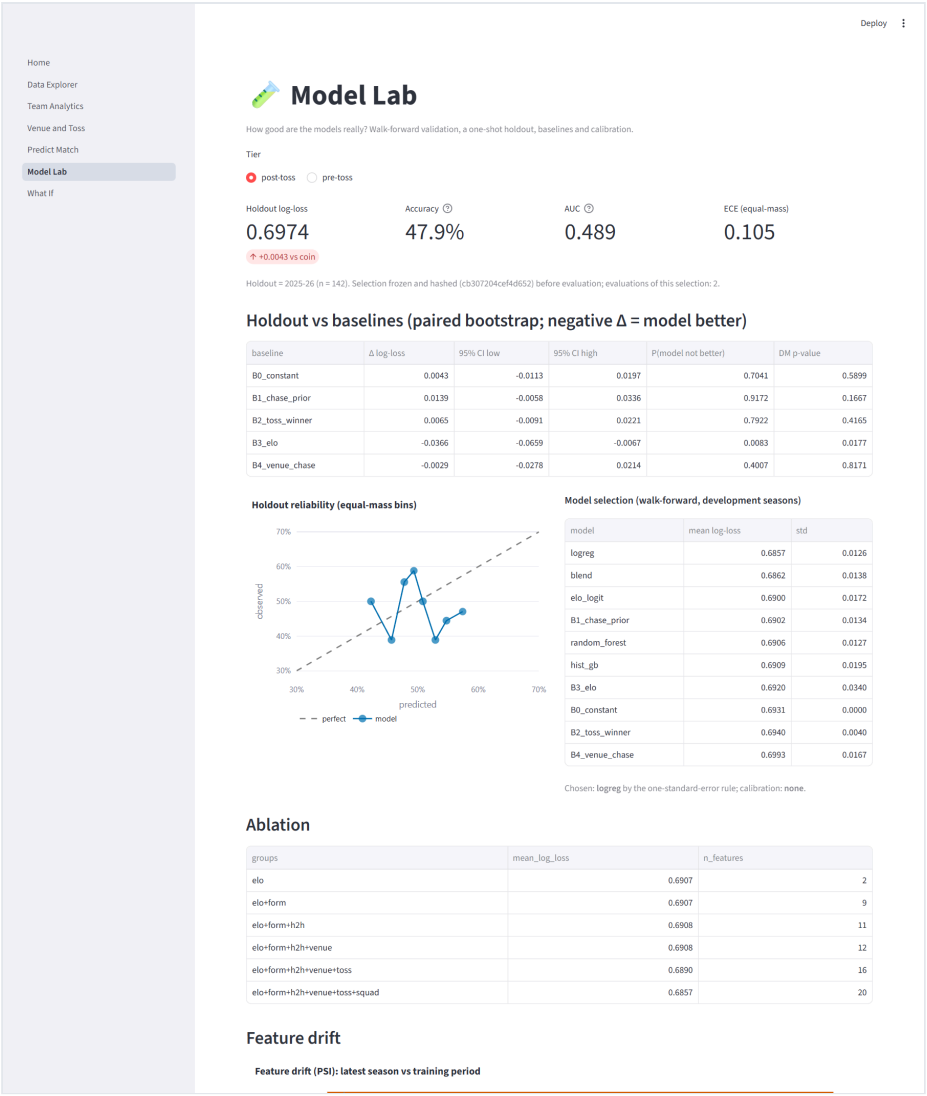


Figure 16: Team Analytics: records with Wilson CIs, titles, season form



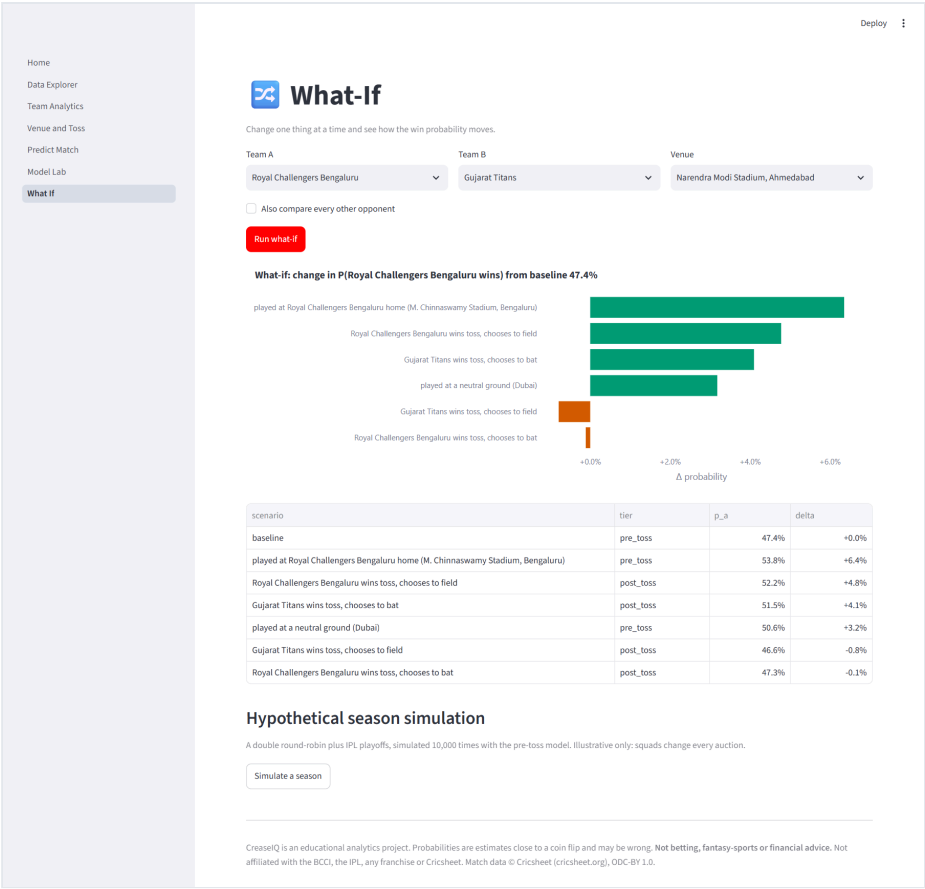


Figure 18: What-If: change in win probability for toss and venue scenarios

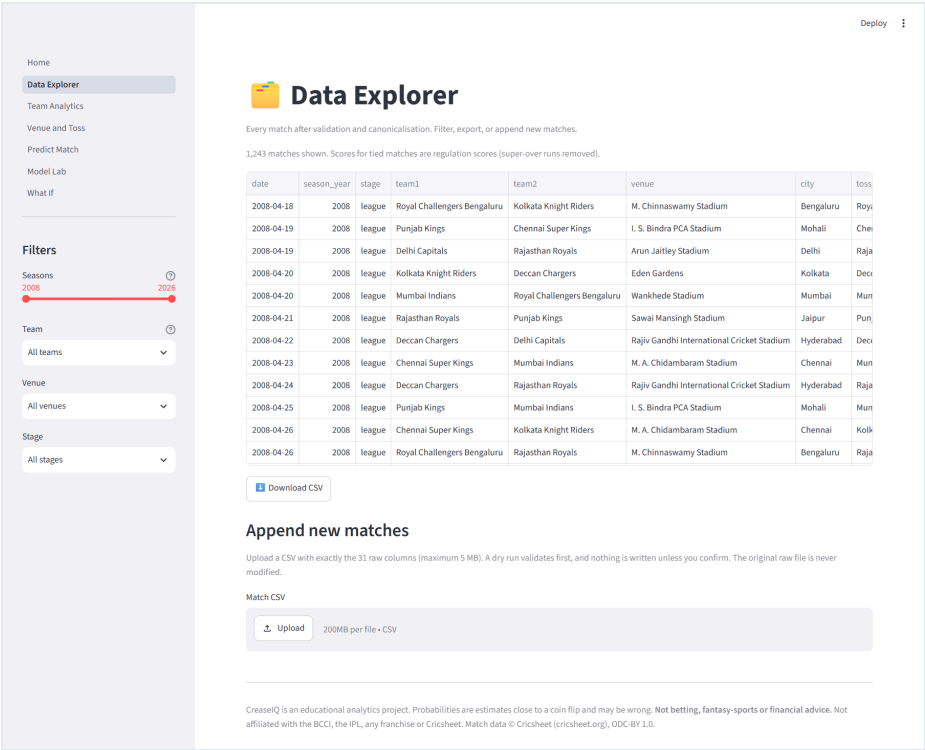


Figure 19: Data Explorer: filters, sanitised export and validated upload

## 10.2 Analytics highlights

Table 16: Headline statistical findings (decided matches)

| Question                                  | Estimate       | 95% CI              | Test                             | Interpretation                         |
|-------------------------------------------|----------------|---------------------|----------------------------------|----------------------------------------|
| Does the toss winner win?                 | 51.6%          | 48.8%-54.4%         | exact binomial $p = 0.29$        | No detectable effect; MDE $\pm 4.0$ pp |
| Does the chasing side win?                | 54.7%          | 51.9%-57.5%         | $p = 0.001$                      | Real, modest chasing edge              |
| Chase edge, Impact era minus pre-2023     | -2.5 pp        | -9.1 to 4.1 pp      | z-test $p = 0.46$                | No evidence it changed                 |
| Home side wins (neutral seasons excluded) | 53.2%          | 50.0%-56.5%         | $p = 0.057$                      | Small, borderline                      |
| First-innings runs, Impact era vs before  | 189.7 vs 162.6 | $\Delta 22.6$ -31.7 | Welch $p = 2.4e-27$ ; $d = 0.82$ | Large effect                           |
| 200+ first innings                        | 41.7% vs 10.1% | n/a                 | n/a                              | 4.1× as frequent                       |

Toss winners choosing to field won 54.7% of matches (45.3% losses; chi-square  $p = 0.002$ ; Cramer's  $V = 0.09$ ). This association may also reflect conditions and team strength.

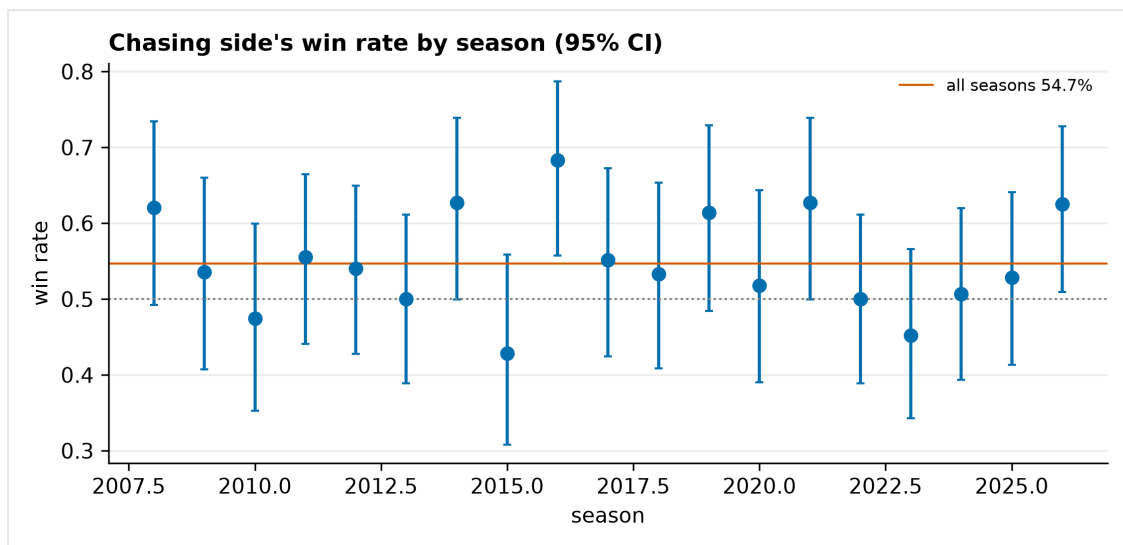


Figure 20: Chasing side's win rate by season with 95% Wilson intervals

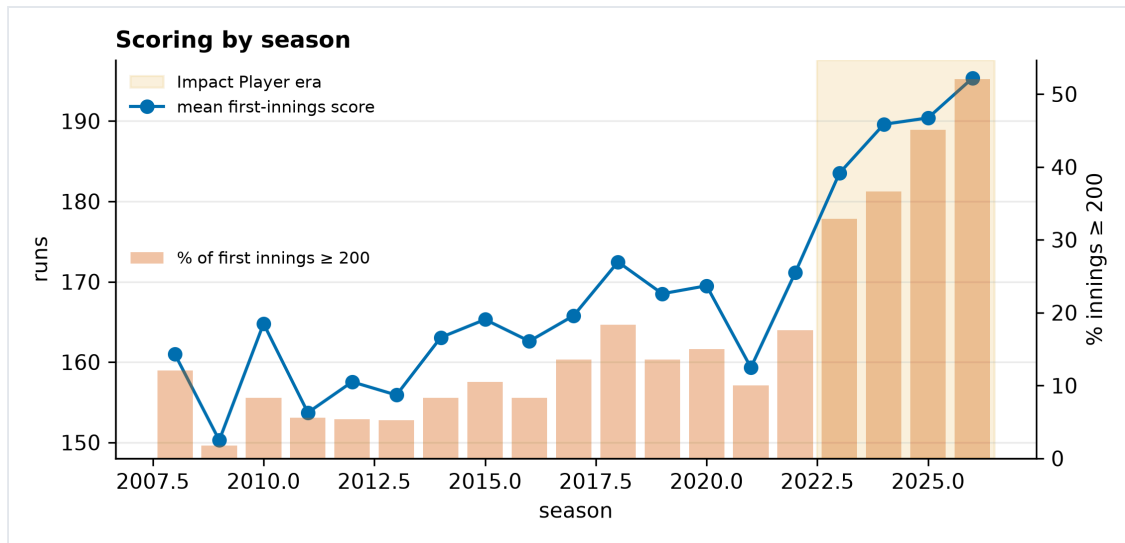


Figure 21: Mean first-innings score and share of 200+ totals; the Impact Player era is shaded

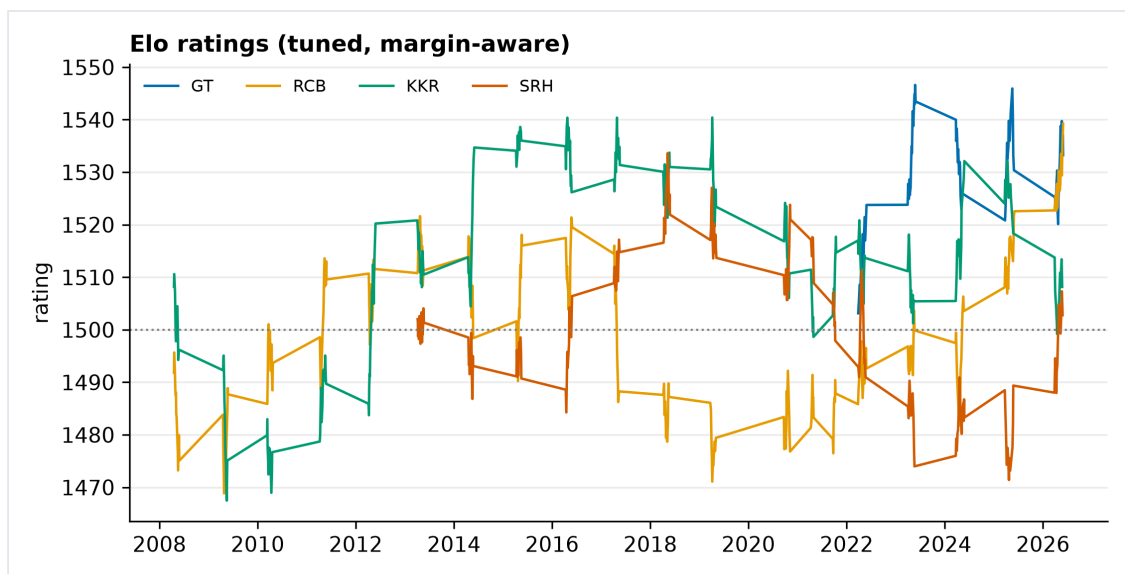


Figure 22: Tuned Elo ratings of the current top franchises

Venue clustering selected two groups using the silhouette score: higher-scoring venues that were less favourable to chasing, and lower-scoring venues that were more favourable to chasing.

### 10.3 Win-probability results

Table 17: Holdout (2025-26) metrics with 95% bootstrap CIs

| Tier      | Model     | Calibration | n   | Log-loss              | Brier  | Accuracy             | AUC                | ECE   |
|-----------|-----------|-------------|-----|-----------------------|--------|----------------------|--------------------|-------|
| post_toss | logreg    | none        | 142 | 0.6974 [0.682, 0.714] | 0.2522 | 47.9% [40.1%, 56.3%] | 0.489 [0.39, 0.59] | 0.105 |
| pre_toss  | elo_logit | none        | 142 | 0.7060 [0.689, 0.723] | 0.2564 | 43.0% [34.5%, 51.4%] | 0.453 [0.35, 0.55] | 0.120 |

Table 18: Holdout comparison with every baseline ( $\Delta$  = model – baseline log-loss; negative means the model is better)

| Tier      | Baseline       | Baseline log-loss | $\Delta$ | 95% CI of $\Delta$ | P(model not better) | DM p |
|-----------|----------------|-------------------|----------|--------------------|---------------------|------|
| post_toss | B0_constant    | 0.6931            | +0.0043  | [-0.0113, +0.0197] | 0.70                | 0.59 |
| post_toss | B1_chase_prior | 0.6835            | +0.0139  | [-0.0058, +0.0336] | 0.92                | 0.17 |
| post_toss | B2_toss_winner | 0.6910            | +0.0065  | [-0.0091, +0.0221] | 0.79                | 0.42 |

| Tier      | Baseline       | Baseline log-loss | $\Delta$ | 95% CI of $\Delta$ | P(model not better) | DM p |
|-----------|----------------|-------------------|----------|--------------------|---------------------|------|
| post_toss | B3_elo         | 0.7340            | -0.0366  | [-0.0659, -0.0067] | 0.01                | 0.02 |
| post_toss | B4_venue_chase | 0.7003            | -0.0029  | [-0.0278, +0.0214] | 0.40                | 0.82 |
| pre_toss  | B0_constant    | 0.6931            | +0.0129  | [-0.0043, +0.0299] | 0.93                | 0.14 |
| pre_toss  | B3_elo         | 0.7340            | -0.0280  | [-0.0487, -0.0068] | 0.01                | 0.01 |

What the results mean. In development validation, the selected post-toss model scored 0.6857 log-loss, ahead of the 50:50 baseline (0.6931) and chase prior (0.6902). Neither tier beat the 50:50 baseline on the 142 holdout matches, and the confidence intervals were wide. We did not retune the models after seeing this result. Home-side win rates were 40%, 58%, 40% and 52% in 2023-2026, compared with 53% overall. These changes suggest that some earlier patterns may not generalise well. The leakage guard was not triggered. The results show the limits of these models on this holdout, rather than a universal limit on IPL prediction.

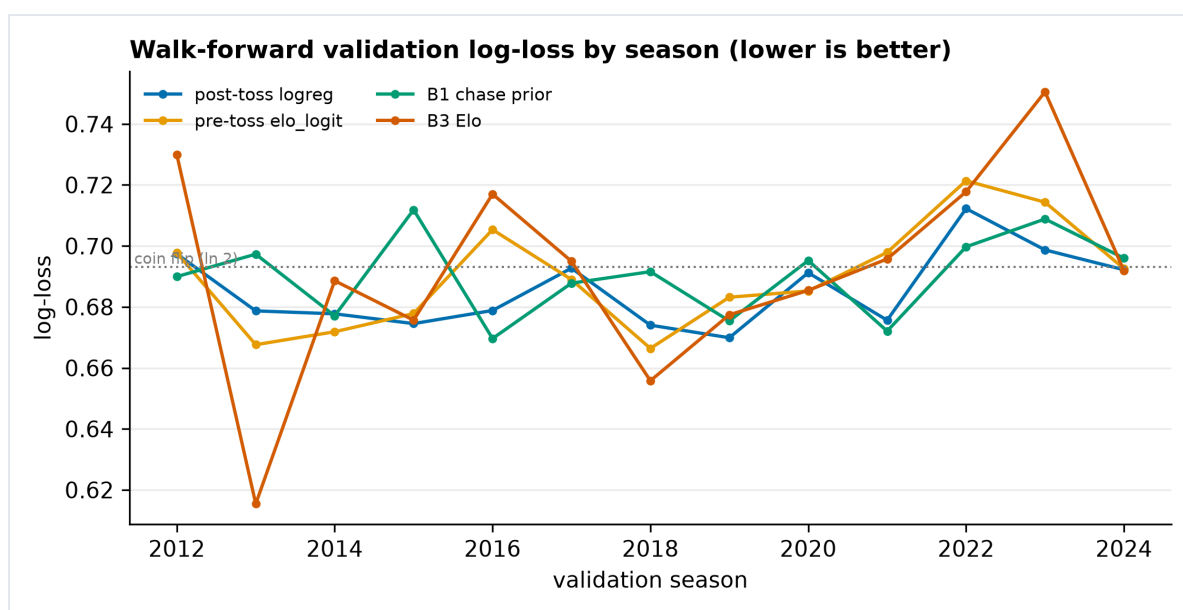


Figure 23: Validation log-loss by season: chosen models against baselines

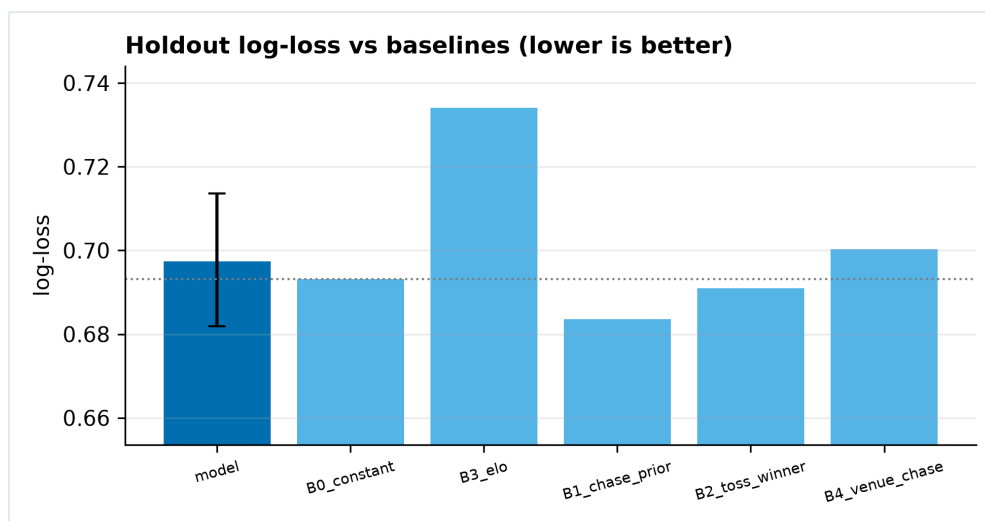


Figure 24: Post-toss holdout log-loss vs baselines (model bar with 95% CI)

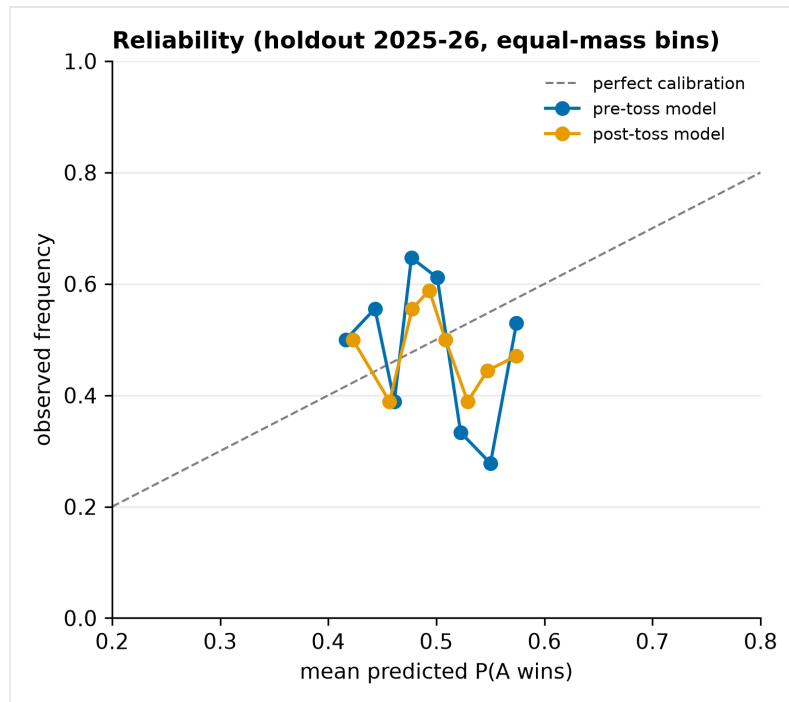


Figure 25: Holdout reliability diagram (equal-mass bins)

Table 19: Cumulative feature-group ablation (post-toss logistic, walk-forward mean log-loss)

| Feature groups                | # features | Log-loss |
|-------------------------------|------------|----------|
| elo                           | 2          | 0.6907   |
| elo+form                      | 9          | 0.6907   |
| elo+form+h2h                  | 11         | 0.6908   |
| elo+form+h2h+venue            | 12         | 0.6908   |
| elo+form+h2h+venue+toss       | 16         | 0.6890   |
| elo+form+h2h+venue+toss+squad | 20         | 0.6857   |

Adding feature groups showed that most of the small improvement came from toss, batting-order and squad information. These are available at the toss. Features based only on match history added little.



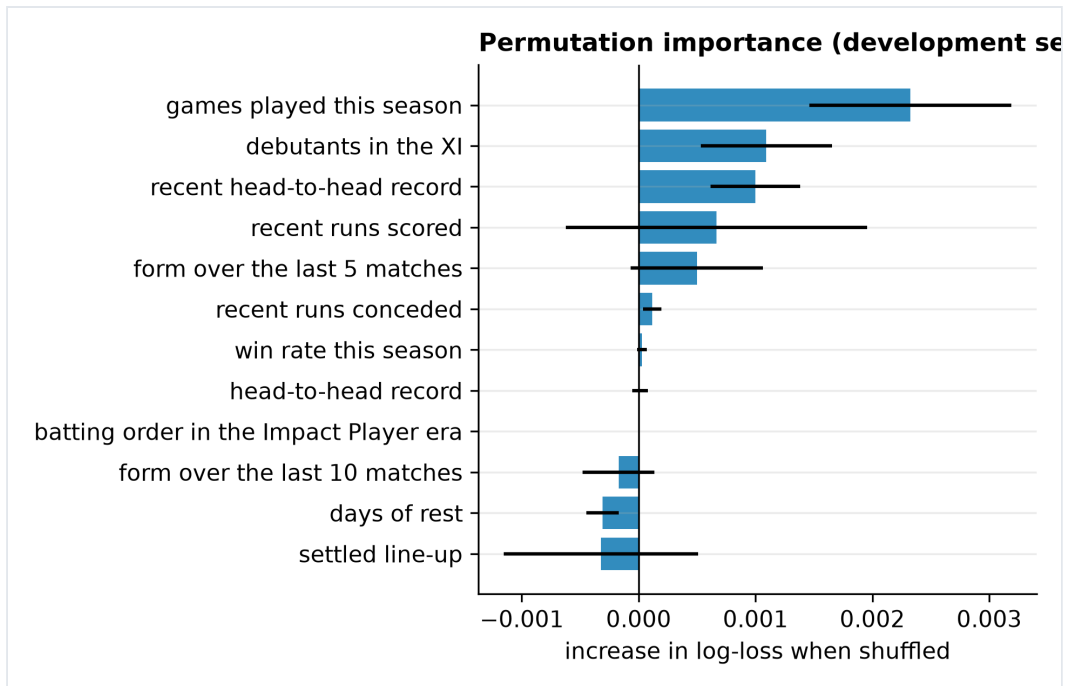


Figure 26: Permutation importance of the post-toss model (development seasons)

Table 20: Validation log-loss by era and stage (time-ordered calibrated out-of-fold predictions, post-toss)

| Segment           | n   | Log-loss | Accuracy |
|-------------------|-----|----------|----------|
| era: impact_2023+ | 144 | 0.6955   | 52.1%    |
| era: pre_2023     | 687 | 0.6847   | 53.3%    |
| stage: league     | 779 | 0.6872   | 52.8%    |
| stage: playoff    | 52  | 0.6776   | 57.7%    |

## 10.4 First-innings score regression

Table 21: First-innings runs, mean absolute error (MAE, runs)

| Model                       | Walk-forward MAE | Holdout MAE [95% CI] | Holdout RMSE | Holdout bias |
|-----------------------------|------------------|----------------------|--------------|--------------|
| ridge                       | 24.7             | 30.0 [26.1, 33.5]    | 37.1         | -3.0         |
| hist_gb                     | 26.3             | 29.9 [26.4, 33.5]    | 37.0         | -4.7         |
| baseline_recent_league_mean | 24.0             | 30.1 [26.3, 33.6]    | 37.3         | -2.6         |
| baseline_venue_level        | 25.1             | 35.6 [31.7, 39.6]    | 43.2         | -22.4        |

The mean score over roughly the last 74 matches was a strong baseline. It adapted to the higher scores in the Impact Player era, while the venue baseline tended to underestimate them. The regression models offered little improvement over the recent league mean.

## 10.5 Drift

The PSI compares the latest season with the development period. The largest shifts are:

- venue\_first\_innings : PSI 5.39 (alert)
- h2h\_meetings : PSI 4.21 (alert)
- bats\_first\_x\_impact : PSI 1.62 (alert)
- bats\_first\_x\_venue : PSI 1.13 (alert)

- `elo_diff`: PSI 1.02 (alert)
- `venue_bat_first_rate`: PSI 0.63 (alert)

Scoring-related features changed substantially after 2023. This may help explain the weaker holdout results and would be a reason to review and retrain the model in production.

# 11. Testing Approach

## 11.1 Test pyramid

Table 22: Collected tests per suite

| Suite                             | Tests | Content                                                                                                              |
|-----------------------------------|-------|----------------------------------------------------------------------------------------------------------------------|
| Unit ( tests/unit )               | 242   | Every data, analytics, feature, model, service and reporting function; Hypothesis property tests; architecture (AST) |
| Integration ( tests/integration ) | 26    | Pipeline on isolated projects; CLI via <code>CliRunner</code> ; <code>Streamlit AppTest</code> for all 7 pages       |
| Validation ( tests/validation )   | 125   | Leakage, reproducibility, experiment, security, documentation, performance                                           |
| Total                             | 393   |                                                                                                                      |

Integration and validation tests use temporary project folders, with their own configuration, external data, raw CSV, database and outputs. This keeps test runs from changing the working repository.

## 11.2 Data-contract tests

- **Raw data:** the real CSV passes strict validation with the documented number of quarantined rows (0).
- **Corruption:** injected faults are caught with named reasons: bad toss decision, non-participant toss winner, 11 wickets on a non-tie, impossible date, same team twice, missing winner, short squad, non-constant overs.
- **Ground truth:** all 19 derived champions match published results; there are 74 playoff fixtures with era-correct labels; base rates are correct; every alias maps.

## 11.3 Leakage tests (written before any model code)

1. **Future deletion:** features for matches up to a cut-off are bitwise identical when all later matches are removed.
2. **Future mutation:** flipping every later result and inflating later scores changes no earlier feature.
3. **Same day:** removing one match of a double-header leaves the other's features unchanged.
4. **Label shuffle:** with shuffled labels, the model's mean AUC is within 0.03 of 0.5.
5. **Column allow-list:** post-match columns raise `FeatureLeakageError`.

In addition:

- The orientation flag alone has no signal (its bootstrap AUC CI contains 0.5).
- A documentation test shows that raw `team1` order *would* leak batting order.
- The "too good" guard fires above AUC 0.72.

## 11.4 Model and property tests

- **Hypothesis properties:**
- Elo expected scores are complementary and monotone.
- Updates are zero-sum and move in the right direction.
- The margin multiplier is positive and increasing.
- Shrunk rates lie between the raw rate and the prior.

- **Swap invariance:**  $|p(A,B) - (1 - p(B,A))| < 1e-9$  for every model family, for calibrated predictions, and for served predictions.
- **Metrics:** cross-checked against scikit-learn and statsmodels (log-loss, Brier, AUC, Wilson, z-test, Holm).
- **Reproducibility:** two full experiment runs give identical selections, grids and holdout metrics (tolerance  $1e-9$ ).
- **Registry:** round-trip loading works; an appended byte triggers `ModelIntegrityError`, and so does a missing artifact.

## 11.5 Security tests

- Every tracked file is scanned for tokens, cloud keys and private keys. This check was added after a token was found in the original build plan and removed before the first commit.
- **SQL:** an AST ban on f-string or concatenated SQL in `text()`, `execute()` and `read_sql()`. Injection strings passed as team ids are inert.
- **Deserialisation:** allowed only in the registry, and only after the hash check. No `eval`, `exec` or `shell=True`.
- **Uploads:** wrong type, oversize, bad encoding, invalid rows and unknown teams are all rejected, with nothing written.
- **Exports:** CSV-injection payloads (`=`, `+`, `-`, `@`, `tab`) are escaped.
- **Static analysis:** `bandit` reports 0 issues and `pip-audit` finds no known vulnerabilities (`reports/bandit.json`, `reports/pip_audit.json`).

## 11.6 Coverage and continuous integration

Table 23: Line coverage (*coverage.py*)

| data  | features | models | analytics | total |
|-------|----------|--------|-----------|-------|
| 98.7% | 99.7%    | 98.9%  | 99.7%     | 94.6% |

The GitHub Actions workflow runs a matrix of  $\{\text{Ubuntu, Windows, macOS}\} \times \{\text{Python 3.12, 3.13}\}$ . Each run executes `ruff check`, `ruff format --check`, `mypy src`, `lint-imports` (the layer contract), `pytest --cov` and `bandit`, plus `pip-audit` once. All gates pass locally on Windows 11, and a clean clone (fresh virtual environment, `creaseiq all`, full test suite) was verified before release. CI runs on every push once the repository is published on GitHub.

## 12. Challenges Faced

---

The build log and ADRs record several practical challenges we encountered:

1. Changing column meaning. We found that team1 always bats first from 2018 onward, although the earlier ordering is effectively random. Looking at each season separately revealed the change. We addressed it by deriving batting order from the toss and assigning team orientation with a hash-seeded coin flip (ADR-005). A test now checks this case.
2. Super-over scores. A record showing 12 wickets lost led us to inspect the tie matches. All 16 included super-over runs and wickets, and one included two super overs. We restored regulation scores after checking two independent sources for each tie (R8).
3. A voided match. PBKS v DC on 08-05-2025 was stopped during a security blackout and later replayed. We flagged it separately so its scores would not be treated like those from a normal no-result match.
4. Missed D/L matches. The margin rule could detect only some rain-adjusted results. A chase won by wickets could still involve D/L, so we combined the rule with a verified external list.
5. Venue identity. We had to distinguish rebuilt stadiums at the same site, different grounds in the same district, and simple name changes. Motera, Mullanpur/Mohali and Sahara/MCA Pune each needed different treatment. An initial count suggested 40 venues, but the completed mapping contains 37; the report uses that mapping.
6. Tool compatibility. The initial plan used Python 3.11, but the selected NumPy and SciPy versions required 3.12. We updated the minimum version (ADR-002) and installed Git and the GitHub CLI before setting up version control.
7. Credentials in a draft. A personal access token appeared in the build plan. We removed it before the first commit, used gh auth login for authentication, and added a secrets scan to help prevent a repeat.
8. **Performance.** Under pandas 3, `itertuples` over about 950 small date groups made one feature build take 7 s, which made grid search and the leakage tests impractical. Profiling showed 117k indexer calls. Converting to plain records once cut the build to 0.33 s, a 21× speed-up.
9. Weak holdout performance. No selected model beat a 50:50 forecast on 2025-26. We kept the frozen selection and reported the result instead of tuning against the holdout. Changes between seasons may be part of the explanation. ADR-006 also records the ledger reset after an exploratory run of the same selection.
10. Problems found during review. The planned chase-rate baseline used all seasons, which would have exposed the holdout base rate. The toss test also conditioned on a later decision. Both were corrected during the 13-point review in docs/plan\_review.md.
11. **Bugs caught by tests and types.** A logging handler wrote to a stream the test runner had closed; Streamlit's logging configuration silently disabled our module loggers; an output writer assumed `docs/` existed; mypy caught a shadowed loop variable in the registry code; and franchise alias validity windows ending in 2026 would have rejected any 2027 upload.
12. **A clean clone failed on Windows.** In the final QA run, a fresh `git clone` failed the raw-data SHA-256 check. Git for Windows ( `core.autocrlf=true` ) had rewritten the CSV's line endings on checkout. A `.gitattributes` file now marks data and model artifacts as byte-exact. The integrity check caught a problem that would otherwise have silently changed the input data.

13. Checking external sources. Some pages blocked automated access, and summaries occasionally disagreed with the underlying material. We used accessible source text and cross-checked the findings. Research notes record the sources and confidence for each external fact.

## 13. Learnings & Key Takeaways

---

1. Understanding the data took the most care. The major issues came from changing column meanings, super-over scores and a voided match. Finding and correcting these problems made the analysis more useful than adding another model would have.
2. Leakage can be easy to miss. Whole-dataset averages, random splits, post-match fields and even team ordering can expose future information. Building features in date order and testing them against deliberately altered data helped us check for these mistakes.
3. Evaluation should reflect the result we actually obtained. Baselines, time-ordered validation and the reserved holdout showed only limited predictive value in the available pre-match features. Earlier studies also discuss uncertainty in T20 cricket [2], [3]. High accuracy claims need careful validation.
4. Probabilities need checking. Accuracy alone cannot tell us whether a 60% prediction is well founded. Log-loss and calibration helped us judge the quality of the probabilities, not just the predicted winners [9].
5. A non-significant result needs context. The toss analysis did not establish a clear effect, but that does not prove the effect is exactly zero. Confidence intervals and the detectable effect size, roughly four percentage points here, help explain what the analysis can support.
6. Consistency can be built into the model. Using antisymmetric features and no intercept made the logistic model consistent when teams A and B were swapped. Automated tests check this property.
7. Simpler models suited this dataset. With roughly 1,000 development matches, the one-SE rule favoured Elo-logit or logistic models over forests and boosting.
8. Reproducibility takes deliberate work. Fixed seeds, pinned dependencies, configuration files, hashes, repeatable database rebuilds and a holdout ledger made results easier to reproduce and check.
9. Shared services made testing easier. The dashboard and CLI use the same project logic, so we could test each operation without opening a browser.
10. Documentation helped us explain the project. Recording the reasons behind decisions made later reviews, report writing and viva preparation easier.

## 14. Future Enhancements

---

1. Ball-by-ball analysis. Add Cricsheet JSON through the existing MatchSource interface. This would support in-play probabilities based on runs, wickets and balls remaining, along with player ratings and powerplay/death-over analysis.
2. Player-level team strength. Use player ratings and auction data to estimate squad strength before a match. This could help the model respond when team composition changes after a mega-auction.
3. Models that adapt across seasons. Compare time-decayed training, explicit era features and hierarchical team-by-season models. Each approach would use the same walk-forward evaluation so the comparison remains fair.
4. Match conditions. Include weather, evening humidity, pitch reports and venue-by-era effects to study how conditions relate to toss decisions and chasing results.
5. Responding to drift. Trigger a review when PSI crosses its alert threshold, then compare a retrained model with the current model before replacing it.
6. API access. Add a REST API, such as FastAPI, over the services layer, with authentication and rate limits if the application supports multiple users.
7. Deployment. Package the read-only dashboard in a container and deploy it with PostgreSQL. The database can already be selected through its URL.
8. Regular data updates. Schedule imports of new fixtures, using the same validation and dry-run checks already provided for uploads.
9. Clearer player identity. Replace exact name matching with Cricsheet registry person IDs to resolve the known ambiguous player name.
10. Accessibility. Check keyboard and screen-reader use, add a high-contrast theme, and provide text descriptions for charts.



## 15. References

---

Research notes in docs/research\_notes.md record source access on 28 September 2026. Where automated access was blocked, the report cites material read through an accessible version. The original report build recorded successful checks for all 14 tested URLs.

- [1] S. Rushe, "Cricsheet: IPL match data (JSON) and Register," cricsheet.org. Available: <https://cricsheet.org/matches/> and <https://cricsheet.org/register/> (Open Data Commons Attribution License v1.0).
- [2] G. Sood and D. Willis, "Fairly random: The impact of winning the toss on the probability of winning," arXiv:1605.08753, 2016.
- [3] S. Kampakis and W. Thomas, "Using machine learning to predict the outcome of English county twenty over cricket matches," arXiv:1511.05837, 2015.
- [4] N. Lamsal and A. Choudhary, "Predicting outcome of Indian Premier League (IPL) matches using machine learning," arXiv:1809.09813, 2018.
- [5] L. M. Hvattum and H. Arntzen, "Using ELO ratings for match result prediction in association football," *International Journal of Forecasting*, vol. 26, no. 3, pp. 460-470, 2010.
- [6] FiveThirtyEight, "nfl-elo-game: forecast.py," GitHub repository. Available: <https://github.com/fivethirtyeight/nfl-elo-game>
- [7] N. Johnsson, "A simple improvement to FiveThirtyEight's NBA Elo model," Harvard Sports Analysis Collective, 2019.
- [8] P. Puram, S. Roy, D. Srivastav and S. Gurumurthy, "Contextual factors and decision making in T20 cricket: an interpretable machine learning approach," *Annals of Operations Research*, vol. 325, 2023.
- [9] C. Walsh and A. Joshi, "Machine learning for sports betting: should model selection be based on accuracy or calibration?," *Machine Learning with Applications*, 2024 (arXiv:2303.06021).
- [10] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192-213, 2012.
- [11] V. Cerqueira, L. Torgo and I. Mozetič, "Evaluating time series forecasting models: an empirical study on performance estimation methods," *Machine Learning*, vol. 109, pp. 1997-2028, 2020.
- [12] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd ICML*, 2005.
- [13] scikit-learn developers, "Probability calibration," scikit-learn User Guide. Available: <https://scikit-learn.org/stable/modules/calibration.html>
- [14] J. Nixon, M. Dusenberry, L. Zhang, G. Jerfel and D. Tran, "Measuring calibration in deep learning," in *CVPR Workshops*, 2019 (arXiv:1904.01685).
- [15] A. Kumar, P. Liang and T. Ma, "Verified uncertainty calibration," in *NeurIPS*, 2019 (arXiv:1909.10155).
- [16] R. Roelofs, N. Cain, J. Shlens and M. C. Mozer, "Mitigating bias in calibration error estimation," in *AISTATS*, 2022 (arXiv:2012.08668).
- [17] "Brier score" (Murphy decomposition), Wikipedia. Available: [https://en.wikipedia.org/wiki/Brier\\_score](https://en.wikipedia.org/wiki/Brier_score)
- [18] Real Statistics Using Excel, "Diebold-Mariano test." Available: <https://real-statistics.com/time-series-analysis/forecasting-accuracy/diebold-mariano-test/>

- [19] S. Bhaskar, "Rational adversaries? Evidence from randomised trials in one day cricket," *The Economic Journal*, vol. 119, no. 534, pp. 1-23, 2009.
- [20] "List of Indian Premier League seasons and results," "2025 Indian Premier League," "2026 Indian Premier League," Wikipedia. Available: [https://en.wikipedia.org/wiki/List\\_of\\_Indian\\_Premier\\_League\\_seasons\\_and\\_results](https://en.wikipedia.org/wiki/List_of_Indian_Premier_League_seasons_and_results)
- [21] Open Data Commons, "Attribution License (ODC-By) v1.0 summary." Available: <https://opendatacommons.org/licenses/by/summary/>
- [22] "Promotion and Regulation of Online Gaming Act, 2025," Wikipedia. Available: [https://en.wikipedia.org/wiki/Promotion\\_and\\_Regulation\\_of\\_Online\\_Gaming\\_Act,\\_2025](https://en.wikipedia.org/wiki/Promotion_and_Regulation_of_Online_Gaming_Act,_2025)
- [23] Python Software Foundation, "Status of Python versions," Python Developer's Guide. Available: <https://devguide.python.org/versions/>
- [24] Streamlit, "st.testing.v1.AppTest," Streamlit documentation. Available: <https://docs.streamlit.io/develop/api-reference/app-testing/st.testing.v1.apptest>
- [25] Microsoft, "Page.pdf," Playwright for Python documentation. Available: <https://playwright.dev/python/docs/api/class-page#page-pdf>
- [26] Cricsheet, "JSON format specification." Available: <https://cricsheet.org/format/json/>
- [27] Vellore Institute of Technology, "Research Integrity Policy." Available: <https://vit.ac.in/files/ebooks/Research-Integrity-Policy/files/basic-html/page7.html>

## Appendix: AI assistance statement

AI assistance (Claude Code) was used for scaffolding, research and review. The author reviewed the design decisions (ADR-001 to ADR-006) and prepared module explanations in docs/viva\_prep.md. Sources and confidence levels for external facts are recorded in docs/research\_notes.md. No code was copied from the IPL projects reviewed for comparison.